ATMx2xx Reference Manual

Revision History

Date	Version	Description
June 28, 2019	0.5	Initial version created.
August 27, 2019	0.51	Updated OTP Protection, Power Modes, Power Management Unit sections, minor formatting update
October 11, 2019	0.52	Updated Related Documentation, PMU, the Clock section under Functional Description, the Receiver section under Modem, Wakeup Receiver sections. Added internal pull up resistor values under Pin Multiplexing section; Peripherals and IO section, minor formatting update
November 7, 2019	0.53	Updated Figure 2.3-3, 2.3-4, 2.3-5, 2.3-6 configuration parameters under Power Management Unit; updated Serial Peripheral Interface (SPI) section under Peripherals and IO
February 14, 2020	0.54	Updated Power Management Unit and Application ADC sections
March 6, 2020	0.55	Updated Related Documentation and Application ADC sections
June 15, 2020	0.56	Added ATM2202, ATM3202, updated SRAM under CPU & Memory, Power Modes and PMU configurations sections, Table 2.10-2
November 23, 2020	0.57	Updated PMU configurations section, Figure 2.3-2, Figure 2.3-4 , Figure 2.3-5, Figure 2.3-8 through Figure 2.3-11,
December 2, 2020	0.58	Corrected typos
March 30, 2021	0.59	Updated Related Documentation, Clocks, Table 2.3 -2 PMU Configuration Fields, Internal or external IO supply, Application ADC added Figure 2.3-5 and Figure 2.3-6 , corrected typos
April 14, 2021	0.60	Updated format, no content change.
July 1, 2021	0.61	Updated Clocks, Voltage Range and CFLAGS Setting.

CONFIDENTIAL

©2019-2022, Atmosic Technologies Inc. All rights reserved. Atmosic logo is a registered trademark of Atmosic Technologies Inc. All other trademarks are the properties of their respective holders. This document is subject to change without notice.

April 6, 2022	0.62	Updated section 2.1.4 Clocks, Table 2.1.4.3-1 APB Subsystem Memory Map, Table 2.1.4.3-2 APB1 Subsystem Memory Map 2.1.9 Power Modes, 2.9 Peripherals and IO, Table 2.10-1 Application ADC Electrical Parameters, Table 2.10-2 Summary of ADC Channels, added support of ATM3231/ATM2231.

CONFIDENTIAL

Table of Contents

1 Product Overview 6
1.1 Related Documentation 6

2 Functional Description 6
2.1 CPU & Memory 8
2.1.1 OTP protection 9
2.1.2 APB interface 10
2.1.3 AHB interface 10
2.1.4 Clocks 10
2.1.4.1 RC Oscillator Measurement 12
2.1.4.2 WRPR (Write Privileged Register) 12
2.1.4.3 APB Subsystem 13
2.1.5 Reset 14
2.1.6 Wakeup Timer 14
2.1.7 General Purpose Timers 14
2,1.8 Interrupts 15
2.1.9 Power Modes 16
2.1.10 Debug Access 17
2.2 Security 17
2.3 Power Management Unit 17
2.4 Sensor Hub 34
2.4.1 Event schedule 34
2.4.2 Data comparison 35
2.4.3 Flash control 35
2.5 Bluetooth LE Radio 36
2.5.1 Link Controller 36
2.5.2 LC Module Descriptions 37
2.5.3 Modem 37
2.5.3.1 Receiver 38
2.5.3.2 Transmitter 38
2.6 Wakeup Receiver 39
2.7 Pin Multiplexing 40

CONFIDENTIAL

2.8 PSEQ 57
2.9 Peripherals and IO 59
2.9.1 Controlling input enable and pull up enable 59
2.10 Application ADC 65
2.11 Register Specification Document 67

CONFIDENTIAL

List of Figures

Figure 2-1 Functional Block Diagram
Figure 2.3-1 Power Management Block Diagram
Figure 2.3-2 Regular Battery, Internal IO Supply, No Harvesting Capability
Figure 2.3-3 Regular Battery, Internal IO Supply, RF Harvesting Source
Figure 2.3-4 Regular Battery, Internal IO Supply, Non-RF Harvesting Source
Figure 2.3-5 Regular Battery, External IO Supply, No RF Harvesting Capability
Figure 2.3-6 Regular Battery, External IO Supply, RF Harvesting Source
Figure 2.3-7 Regular Battery, External IO Supply, Non-RF Harvesting Source
Figure 2.3-8 No Battery, Internal IO Supply, RF Harvesting Source
Figure 2.3-9 No Battery, Internal IO Supply, Non-RF Harvesting Source
Figure 2.3-10 Rechargeable Battery, External IO Supply, RF Harvesting Source
Figure 2.3-11 Rechargeable Battery, External IO Supply, Non-RF Harvesting Source
Figure 2.3-12 Rechargeable Battery, Internal IO Supply, RF Harvesting Source
Figure 2.3-13 Rechargeable Battery, Internal IO Supply, Non-RF Harvesting Source
Figure 2.5-1 Bluetooth LE Radio Block Diagram
Figure 2.6-1 Wakeup Receiver Block Diagram
Figure 2.10-1 Application ADC block diagram

List of Tables

Table 2.1-1 System memory map
Table 2.1.4.3-1 APB Subsystem Memory Map
Table 2.1.4.3-2 APB1 Subsystem Memory Map
Table 2.3-1 PMU External Pins
Table 2.3-2 PMU Configuration Fields
Table 2.3-3 PMU Supported Configuration
Table 2.3-4 VBAT Brownout Threshold Level and CFLAGS Setting
Table 2.3-5 Voltage Range and CFLAGS Setting
Table 2.7-1 Multiplexed Pin Function
Table 2.7-2 Multiplexed Pin Function, cont'
Table 2.7-3 Pin Selection Control to Pin Mapping
Table 2.7-4 Pin Selection Control and IO Configuration (0-7)
Table 2.7-4 Pin Selection Control and Corresponding IO Configuration (8-15)
Table 2.7-4 Pin Selection Control and IO Configuration (16-23)
Table 2.9-1 Interrupt Registers
Table 2.10-1 Application ADC Electrical Parameters
Table 2.10-2 ADC Summary of ADC Channels

CONFIDENTIAL

1 Product Overview

The ATM3201/ATM3221, ATM2201/ATM2221, ATM3231/ATM2231 and ATM3202/ATM2202 are part of a family of extreme low-power Bluetooth® 5.0 system-on-a-chip (SoC) solutions. This Bluetooth Low Energy SoC integrates a Bluetooth 5.0 compliant radio with ARM® Cortex® M0 application processor, 128 KB Random Access Memory (RAM), 256 KB Read Only Memory (ROM), 4 KB One-Time-Programmable (OTP) memory, and state-of-the-art power management.

In addition, the ATM3202/ATM2202 supports 128 KB, 512 KB or 1 MB embedded Flash.

This reference manual describes in detail on how to implement solutions using ATM3201/ATM3221/ATM3231, ATM2201/ATM2221/ATM2231, ATM3202 and ATM2202.

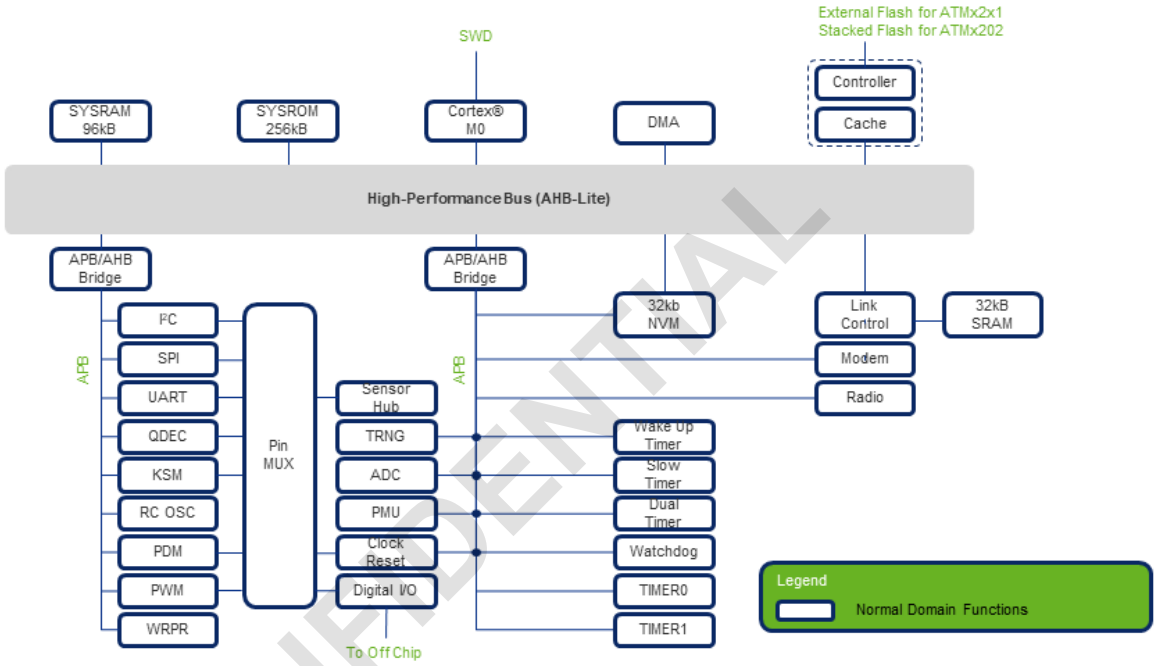
1.1 Related Documentation

- ARM Cortex-M0 Technical Reference Manual available from https://www.arm.com
- Atmosic ATM3201/ATM3221/ATM3231 Datasheet
- Atmosic ATM2201/ATM2221/ATM2231 Datasheet
- Atmosic ATM3202 Datasheet
- Atmosic ATM2202 Datasheet
- Atmosic ATMx221 EVK User's Guide
- Atmosic ATMx201 EVK User's Guide
- Atmosic EVK Quick Start Guide Power Consumption Evaluation
- Atmosic SDK Quick Start Guide Configure Beacon Parameters in OTP
- Atmosic ATM32xx Register Specification
- Atmosic ATM3xxx Sensor Hub White Paper
- Atmosic Bluetooth LE Coded PHY White Paper
- Atmosic Pin Assignment and Pinmux Programming White Paper
- Atmosic Understanding ATM2/3 Low Power Modes White Paper
- Atmosic ATM2/ATM3 Wakeup Receiver White Paper
- Atmosic ATM32x1 Energy Harvesting Application Notes
- Atmosic ATM3 Energy Harvesting White Paper
- Atmosic ATM32x1 EVK Energy Harvesting Quick Start Guide
- Atmosic PIN MUX Software Tool
- Atmosic Chip Revisions Application Note

CONFIDENTIAL

2 Functional Description

Figure 2-1 Functional Block Diagram



CONFIDENTIAL

2.1 CPU & Memory

The ATM2/ATM3 contains a 16 MHz 32-bit ARM® Cortex®-M0 processor that is optimized for low-power operation. The processor is a little endian, 32-bit RISC processor which implements the ARMv6-M architecture specification. It supports most Thumb-1 instructions and some Thumb-2 instructions. It features four breakpoints, a serial access debug port, 32 interrupts, a single cycle multiplier, full wake-on-interrupt support, and two watch points.

The on-chip system memory is organized as follows. Details of the registers are documented in the Register Specification file.

Table 2.1-1 System memory map

Start	Stop	Block
0x0000 0000	0x0003 FFFF	ROM
0x1000 0000	0x10FF FFFF	FLASH
0x2000 0000	0x2001 7FFF	SRAM
0x3000 0000	0x3000 FFFF	LC
0x4000 0000	0x4000 FFFF	APB0_Peripherals
0x4001 0000	0x4001 1FFF	GPIO
0x4001 F000	0x4001 FFFF	SYSCTL
0x5000 0000	0x5000 FFFF	APB1_Peripherals
0x6000 0000	0x6000 0FFF	NVM
0x7000 0000	0x7000 0FFF	DMA
0x8000 0000	0x8000 0FFF	PDM

CONFIDENTIAL

The ATM2/ATM3 includes the following memory components. In general, each core is compliant with AMBA AHB Protocol Specification v2.0 or AMBA APB Protocol Specification v1.0.

- ROM: 256 KB of ROM containing the Bluetooth LE 5 protocol stack up to GATT/GAP layer, a beacon application, and the boot code. The physical implementation consists of two 128 KB macros. Macro 0 is at the lower address space and macro 1 is at the higher. Each macro has its own supply head switch.
- External FLASH: A quad Serial Peripheral Interface (SPI) master port to interface with an external flash chip for the ATM3201/ATM3221/ATM3231/ATM2201/ATM2221/ATM2231 if needed (up to 16 MB)
- Embedded FLASH: ATM3202/ATM2202 supports 128 KB, 512 KB or 1 MB embedded flash for customer applications.

■ SRAM: 128 KB of SRAM containing both system RAM and data RAM organized as eight 16 KB macros. The power state of each macro in each low power state can be independently controlled. Two of the eight macros are reserved for the Bluetooth 5 link controller while six of the 16 KB macros are available for system RAM as an AHB slave with a total 96 KB of general purpose SRAM. Macro 0 is at the lowest address space and macro 6 is at the highest. Each macro has its own supply head switch which enables a per macro optimization option when going into the Retention low power state.

■ DMA: DMA provides a way to copy a contiguous block data from a source address to a target address. In copy use case, the user must specify the source address, **src_addr**[1], the target address, **tar_addr**, and size of data, size to prime the finite state machine (FSM). Then user must write one to **opmode**.*go* to activate FSM into action and immediately write zero to **opmode**.*go*[2].

■ NVM: 4 KB of One-Time-Programmable memory (OTP) to store configuration, calibration data and user data. The OTP is accessible through two separate options:

(a) APB bus provides indirect access for byte reads and bit writes

(b) AHB bus provides memory mapped addresses for byte, half word, and word reads.

The e-fuse macro (NVM) is accessible through two cores; at_apb_nvm and at_ahb_nvm. The core at_apb_nvm is an APB slave, has both read and program functionality, and offers indirect access. The user must explicitly specify which of the 32,768 e-fuses is being targeted. The core at_ahb_nvm is an AHB slave, has read functionality only, and provides a memory-mapped interface to the NVM. An e-fuse returns 1'b0 when not programmed and 1'b1 after programming. Programming is not reversible.

A hardware FSM interfaces with the e-fuse macro and all timing parameters associated with the e-fuse macro are adjustable via software writes to the NVM **t_**x registers. Units are clk_mpc cycle counts. The default values of these registers are valid for the e-fuse macro and it is not anticipated that their values will need to be changed.

2.1.1 OTP protection

The last word in NVM (the word at the highest address) has a predefined meaning. It contains configuration and protection settings and is automatically read by the hardware core, at_ahb_nvm, at the start of the chip's power-on reset. The value of this word is CPU accessible via a read to **efuse_autoread** register.

Two of the bits in efuse_autoread enable HW protection that may be of interest in certain applications. One of these bits disables the Serial Wire Debug (SWD) interface and the other prevents subsequent writes to the OTP from actually changing the state of any fuse within the OTP. Disabling SWD access is significant because it disables the only interface in which the SoC is the slave. Without SWD, an external device cannot access the internal memory space and any critical data, such as keys, are protected from external queries. Note that setting an e-fuse is not reversible.

To set the SWD disabling e-fuse, set NVM Address Register to 25,596 and then initiate the programming operation. When using the SDK, use the following commands to program the appropriate OTP bit:

make pull_otp_nvds

make rebuild_otp_nvds otp_nvds.data= otp_nvds.hw=SWD_DISABLE

make push_otp_nvds

[1] Text in bold is register name

[2] Text is italics is register field name

CONFIDENTIAL

To set the OTP write blocking e-fuse, set NVM Address Register to 31,743 and then initiate the programming operation. When using the SDK, use the following commands to program the appropriate OTP bit:

make pull_otp_nvds

make rebuild_otp_nvds otp_nvds.data= otp_nvds.hw=DISABLE_OTP_WRITES

make push_otp_nvds

2.1.2 APB interface

To program an e-fuse the user must specify the address of the e-fuse of interest in NVM **address** register and write 1'b1 to **opmode** register, program and go bits (**opmode**.*program* and **opmode**.*go*). The busy status is indicated in the **status** register.

Reading from the e-fuse is byte based so only the lower 12 bits of address are used. The user can also specify how many bytes to read (up to 4) using **opmode**.*wordsize*. Once **address** and **opmode**.*wordsize* are programmed, then write 1'b1 to **opmode**.*read* and **opmode**.*go*. While the FSM is in progress, the status is indicated in status register. Once the read is completed, the data is available for CPU read in **read_data** register. Furthermore, the core also offers an interrupt mechanism to alert the CPU when the FSM read is completed. The relevant registers are **interrupt_mask** for on/off selection, and **interrupt_status** for status. **opmode**.*auto_inc* can be set if auto address incrementing is desired. Use of **opmode**.*auto_inc* will reduce the number of register writes when operating in burst mode.

Each read from the e-fuse macro is byte-wise. When more than one byte is specified for a read, the hardware controller will issue multiple byte reads until the correct number of bytes is reached. The e-fuses associated with a byte read are not contiguous in the address space.

The following pseudo code shows how a particular read address can be mapped to either e-fuse addresses to be written:

```
die if ($byte_addr < 0 || $byte_addr > 4096);
  # which 8kb macro is it in?
  if    ($byte_addr >= 0    && $byte_addr < 1024) { $macro = 0; }
  elsif ($byte_addr >= 1024 && $byte_addr < 2048) { $macro = 1; }
  elsif ($byte_addr >= 2048 && $byte_addr < 3072) { $macro = 2; }
  else                                            { $macro = 3; }

  $base = 8*1024*$macro;
  $offset = $byte_addr - 1024*$macro;

  # now display the e-fuse address to program from MSB to LSB
  for ($bitPos = 7; $bitPos >= 0; $bitPos--) {
      $efuse = 1024*$bitPos + $base + $offset;
      print $efuse
  }
```

Refer to the Atmosic SDK Quick Start Guide Configure Beacon Parameters in OTP and EVB user's guide for information on OTP programming.

2.1.3 AHB interface

The AHB core only has read functionality. The user accesses the NVM data via a memory-mapped interface. The core at_ahb_nvm takes an AHB read access and triggers the read FSM inside at_apb_nvm with the appropriate address and word size.

2.1.4 Clocks

There are three primary clock domains on the ATM2/ATM3: low power clock (CLK_LPC), the medium power clock (CLK_MPC), and the high power clock (CLK_HPC). CLK_LPC is nominally 32.768 kHz, CLK_MPC is 16 MHz, and CLK_HPC is 32 MHz.

CONFIDENTIAL

- CLK_LPC is used to sequence transitions between low power and active states and to detect wake events. CLK_LPC can be derived from either a crystal based circuit or an RC oscillator based circuit, RCOSC. ATMx2xx-x0x and ATM32xx-x1x applications that require a Bluetooth LE connection or tracking real time accurately must have an external 32.768 kHz crystal. All ATM22xx-x1x applications can operate without an external 32.768 kHz crystal. Applications that require a Bluetooth LE connection can be built with the LPC_RCOS makefile argument with the following limitations:

  1. Temperature:
  - The default software settings are valid for an operating ambient temperature range of -20 to 85 C. Please contact Atmosic Support to discuss applications that need to operate below -20 C without an external 32.768 kHz crystal.
  - The default software settings are valid for a maximum temperature drift of 1 degree Celsius every 5 seconds. Please contact Atmosic Support for instructions to modify the settings for applications that need to support faster temperature drift.

  2. Current Consumption:
  Operating without an external 32.768 kHz crystal affects the typical average current consumption during Bluetooth LE activity in the following ways:

  1) <1 µA increase in the retention current to reduce the RCOSC frequency variation
  2) <2 µA increase to periodically measure the RCOSC frequency based on the default software settings. For applications that support a faster temperature drift, this increase will be larger.
  3) <5 µA increase to account for RCOSC frequency variation by waking early for each Bluetooth LE event
  4) Increase to the fixed overhead for each Bluetooth LE event (impact depends on the event interval)

  Increasing the event interval reduces the current consumption impact. For example, operating without an external 32.768 kHz crystal would increase the typical average current consumption during an idle Bluetooth LE connection from ~17 µA to ~27 µA for a 100 ms interval and from ~5 µA to ~9 µA for a 1 sec. interval.

  3. Timing Accuracy
  - Applications that require tracking real time accurately need to account for RCOSC frequency variability. Please contact Atmosic Support for additional support.

  If there is no external 32.768 kHz crystal, set NVM Address Register to 32,767 and then initiate the programming operation. When using the SDK, use the following commands to program the appropriate OTP bit:
  make pull_otp_nvds
  make rebuild_otp_nvds otp_nvds.data= otp_nvds.hw=NO_32KHZ_XTAL_ON_BOARD
  make push_otp_nvds

- CLK_MPC is the largest of the three clock domains. CLK_MPC can be derived from either a crystal based circuit or an RC oscillator circuit. An external crystal for CLK_MPC is required for the radio to operate properly. The RC oscillator may be used when the radio is not in use to conserve power. By default the transitions between the crystal based circuit and the RC oscillator based circuit for CLK_MPC are done automatically by the module PSEQ. When the Bluetooth core notifies PSEQ that either a receive or transmit is imminent, the clock source will be switched over to crystal based. When the radio event is over, the clock source will be switched back to RC oscillator based. The user can override this behavior if needed.

- CLK_HPC is used by the quad SPI module to reduce read latency of the flash.

CONFIDENTIAL

2.1.4.1 RC Oscillator Measurement

The core at_apb_rcos_cal provides a way to find the frequency of an imprecise clock (RC oscillator). It takes 2 clocks, one at 32 kHz (slow) and the other at 16 MHz (fast), as input and run two separate counters on each clock for a duration. Of these two clocks, one is the reference and the other imprecise. At the end of the run, the user finds the unknown frequency by the equation : F_unkown = F_ref * Count_unknown/Count_ref. The variables represent the following quantities:

1. F_unknown: unknown frequency of the imprecise clock.
2. F_ref: frequency of the reference clock.
3. Count_unknown: the value of the counter that runs on the imprecise clock.
4. Count_ref: the value of the counter that runs on the reference clock.

Measurement can be manually activated by the software if **cal_ctrl**.*hw_cal_sel* is set to 0 or automatically triggered by the hardware. In the software-controlled mode (**cal_ctrl**.*hw_cal_sel* = 0), the user can set the duration of the run in terms of the slow clock cycles in **slow_clk_set** register and trigger the counting by setting **cal_ctrl**.*sw_cal* to one. The ongoing status is indicated by **stat**.*running*. Once the counting is completed, the values of the 2 counters are available in **slow_clk_cnt** and **fast_clk_cnt** registers.

One use case is to find the frequency of the 32 kHz generated by the RC oscillator. The user then must program to have the 16 MHz clock input to the core be sourced by the XTAL clock. In this case, the variables in the equation are:

1. F_unknown: frequency of the 32 kHz RC clock.
2. F_ref: frequency of the XTAL 16 MHz.
3. Count_unknown: slow_clk_cnt because the 32 kHz (slow) clock is the imprecise clock.
4. Count_ref: **fast_clk_cnt**

Another use case is to find the frequency of the 16 MHz generated by the RC oscillator. The user then must program to have the 16 MHz clock input to the core be sourced by the RC oscillator. Now, the reference clock is the slow clock. If the slow clock also comes from an RC oscillator, then the user must find the frequency of the slow clock first. Once the slow clock is known, the variables in the equation are as follows:

1. F_unknown: frequency of the 16 MHz RC clock.
2. F_ref: frequency of the slow clock.
3. Count_unknown: **fast_clk_cnt** because the fast clock is the imprecise clock.
4. Count_ref: **slow_clk_cnt**.

In the hardware-controlled mode (**cal_ctrl**.*hw_cal_sel* = 0), each core's run is enabled when the signal "rif_rf_busy" is high, which implies that the source of the 16 MHz clock to the core is XTAL. This mode is strictly used to calculate the frequency of the slow clock, assuming the source of the 32 kHz is RC oscillator. In this mode, the software only needs to read the values of the counters and apply the equation to find the frequency of the slow clock. The duration of the counting is dictated by the "rif_rf_busy", unlike the software-controlled mode in which the counting duration is set by **slow_clk_set**.

2.1.4.2 WRPR (Write Privileged Register)

The WRPR core contains registers for clock gating, for reset forcing, and for configuration settings of cores that are not directly CPU accessible. Clock gating and reset forcing functionality is limited to the APB cores located on the APB bus. By default each downstream core is in reset and is clock gated off. To enable a downstream core, the recommended approach is to enable clock and then disable reset in separate APB writes.

The core contains the PINMUX configuration registers (**pin_selection_A** through F, **pin_pu_A** through E). This is where the assignment between physical pin and functional core is made. Each physical pin can be configured independently. Not all SoC functions are mapped to all physical pins.

CONFIDENTIAL

The core contains the interrupt masks. This is where the user can configure which collection of interrupt sources are combined into each interrupt. The core also contains both the chip ID field and the chip revision field.

2.1.4.3 APB Subsystem

The APB0 subsystem has 16 APB slave cores. It contains a bridge to and from the AHB bus and concatenates the interrupts of the underlying APB slaves. The cores spi0 and spi1 are meant to communicate with an off-chip SPI slave device. The core spi2 is one of three possible masters of the quad SPI interface.

Table 2.1.4.3-1 APB Subsystem Memory Map

Start	Stop	Block
0x4000 0000	0x4000 0FFF	TIMER0
0x4000 1000	0x4000 1FFF	TIMER1
0x4000 2000	0x4000 2FFF	DUALTIMER
0x4000 3000	0x4000 3FFF	WRPR
0x4000 4000	0x4000 4FFF	UART0
0x4000 5000	0x4000 5FFF	UART1
0x4000 6000	0x4000 6FFF	PWM
0x4000 7000	0x4000 7FFF	SPI0
0x4000 8000	0x4000 8FFF	WATCHDOG
0x4000 A000	0x4000 AFFF	KSM
0x4000 B000	0x4000 BFFF	SPI2
0x4000 C000	0x4000 CFFF	QDEC
0x4000 D000	0x4000 DFFF	SPI1
0x4000 E000	0x4000 EFFF	SLWTIMER
0x4000 F000	0x4000 FFFF	QSPI

The APB1 system has 16 APB slave cores. It contains a bridge to and from the AHB bus and concatenates the interrupts of the underlying APB slaves.

Table 2.1.4.3-2 APB1 Subsystem Memory Map

CONFIDENTIAL

Start	Stop	Block
0x5000 0000	0x5000 0FFF	I2C0
0x5000 1000	0x5000 1FFF	I2C1
0x5000 2000	0x5000 2FFF	NVM
0x5000 3000	0x5000 3FFF	WRPR1
0x5000 5000	0x5000 5FFF	RADIO
0x5000 7000	0x5000 7FFF	PMU
0x5000 8000	0x5000 8FFF	PSEQ
0x5000 A000	0x5000 AFFF	GADC
0x5000 B000	0x5000 BFFF	TRNG
0x5000 C000	0x4000 CFFF	RCOS_CAL
0x5000 D000	0x5000 DFFF	SHUB
0x5000 F000	0x5000 FFFF	PDM

CONFIDENTIAL

2.1.5 Reset

The PMU releases the chip-wide reset once the power supplies have stabilized. There is no explicit reset pin on the ATM2/ATM3 but the user can use PWD for an equivalent purpose. Many of the internal modules can additionally be reset through a software register write.

2.1.6 Wakeup Timer

Wakeup timer is a 40-bit timer based on the low-power 32 kHz clock. When this timer is enabled during SoC Off mode, it will determine the SoC Off duration.

2.1.7 General Purpose Timers

There are four general purpose timer cores: Timer0, Timer1, Dual Timer, and Slow Timer. The value of the timers are readable by the CPU. All timers are clocked by the medium power clock (CLK_MPC) except for the Slow Timer which uses the low power clock (CLK_LPC). All these timers stop when the system enters a low power mode. The state of the timer is maintained during retain mode but is reset during hibernate mode.

- Timer0 on APB bus, if enabled, will decrement from a 32 bit reload value to 0 triggering a maskable interrupt as it transitions from 1 to 0. The reload value is loaded as the next timer value when the timer reaches 0.

- Timer1 is identical to Timer0.

- Dual Timer on APB bus contains two timers and each timer is independent of the other but sharing a single interrupt output. Each timer counts down and can run in one-shot, periodic, or free running mode. The timers are configurable to be either 16 bit or 32 bit. Additionally there is a rescale in front of each timer that can reduce the incoming clock by 1x (no change), 16x, or 256x.

- Slow Timer on APB bus is a 40-bit count down timer with three programmable thresholds. The interrupt is optionally asserted when the counter counts from one above the threshold to the threshold. It is recommended that the timer be read twice and the value used only if the values from both reads are the same. If the values are not the same then the read should happen a third time. The timer is updating approximately every 488 cycles of the 16 MHz medium power clock.

2,1.8 Interrupts

The SoC supports the following interrupts:

- 1 non-maskable interrupt: Watchdog interrupt based on ARM's Cortex-M System Design Kit Rev r1p0. The 32-bit down-counter is initialized from the **WDOGLOAD** register. The counter decrements by one on each positive edge of the clock while the watchdog clock is enabled. The watchdog module generates a regular interrupt, **WDOGINT**, when the counter reaches 0. The watchdog monitors the interrupt and asserts a reset request WDOGRES signal when the counter reaches 0, and the counter is stopped. On the next enabled watchdog clock edge, the counter is reloaded from the **WDOGLOAD** register and the countdown sequence continues. If the interrupt is not cleared by the time the counter next reaches 0, the watchdog module reasserts the reset signal. The watchdog module applies a reset to a system in the event of a software failure, providing a way to recover from software crashes. You can enable or disable the watchdog unit as required.

- 32 maskable interrupts: Upper 16 interrupts are the GPIO interrupts. Lower 16 interrupts are the user programmable combining of the 48 possible interrupt sources. Each of these 16 user configurable interrupts has its own combining function. The maskable interrupt sources are as follows:
    - gpio0_intr, // 16b -> [47:32]
    - pmu_interrupt, // 1b -> [31]
    - pseq_intrpt, // 1b -> [30]
    - dma_done, // 1b -> [29]
    - dma_err, // 1b -> [28]
    - ble_interrupt, // 1b -> [27]
    - atlc_interrupt, // 1b -> [26]
    - gpio1_combintr, // 1b -> [25]
    - gpio0_combintr, // 1b -> [24]
    - apbsubsys1_interrupt, // 8b -> [23:16]

```
apbsubsys_interrupt = {
                      1'b0,
                      1'b0,
                      i_spi2_int,
                      i_slwtimer_int,
                      i_ksm_int,
                      i_uart1_overflow_int,
                      i_uart0_overflow_int,
                      i_spi0_int,
                      i_dualtimer2_int,
                      i_timer1_int,
                      i_timer0_int,
                      i_spi1_int,
                      i_uart1_txint,
                      i_uart1_rxint,
                      i_uart0_txint,
```

CONFIDENTIAL

```
                    i_uart0_rxint};
```

- apbsubsys_interrupt // 16b -> [15:0]

```
 apbsubsys1_interrupt = {
                    i_trng_int,  // 7
                    i_gadc_int,  // 6
                    i_pdm_int,          // 5
                    i_nvm_int,   // 4
                    i_i2c1_int,         // 3
                    i_i2c0_int,         // 2
                    i_pmu_int,   // 1
                    i_radio_int};       // 0
```

2.1.9 Power Modes

The SoC supports five primary power states which are Active, Retention, Hibernation, SoC Off and Powerdown. Each primary state may have several secondary states depending on the number of active power domains and clock gating. Refer to the Understanding ATMxxxx Low Power Modes White Paper for additional details.

1. Active: All regions of the SoC are powered on. Active power can be optimized by utilizing clock gating registers and/or by putting the Cortex-M0 into Wait for Interrupt (WFI).
    a. CPU Idle: When the CPU function is not required, it can be placed into WFI state to conserve power.
    b. Bluetooth LE Deep Sleep: Bluetooth subsystem is powered down while the remainder of the ATM2/ATM3 is powered up. This state is useful when data needs to be processed but does not need to be transmitted over RF.

2. Retention: All or some of the 128 kB SRAM, in increments of 16 kB, can be retained. All register/flip-flop states are retained. Digital IO's will hold the state they were at when the transition into either Hibernate or Retain started. Wake can be from a timer expiring, activity detected on GPIO's, activity detected on the keyboard, activity detected on the mouse, the sensor hub reading measurements crossing a threshold, and the detection of a connection over the SWD interface. All selections about how to wake need to be programmed before the transition into the low power state is triggered. The SRAM supply voltage can be lowered to further reduce leakage power consumption.

3. Hibernation: Powers down system memory. Retains only a minimal amount of flip-flop state. Retains IO state. Wake up setting must be programmed before transitioning into this state.

4. SoC Off: all digital domains including the top level digital domain are powered down, but the PMU remains on in an ultra low-power state with limited functionality. Setting PSEQ **ctrl0**.*req_soc_off* to 1'b1 requests the SoC to enter SoC Off mode as soon as possible. The system must do a complete, cold start reboot when returning from this state to an active state. Wake mechanisms are limited to (a) special 40-bit timer, (b) external pin edge on P10, (c) ultra-low power analog comparator with input on either P11 or P12.
    a. To wake on timer, the desired powered down duration is written into the 40-bit value in PMU_TOP **pmu1**.*timer_target_msb* and **pmu0**.*timer_target_lsb*. A wake event is initiated when this timer crosses from 1 to 0. Additionally **pmu1**.*how2wkup*[2] is set which denotes that (at least) wake on timer will be the wake mechanism.
    b. To wake on GPIO activity, only **pmu1**.*how2wkup*[0] needs to be set to denote wake on GPIO activity. The wakeup circuitry will check for a high level on P10.. This functionality is only valid with an external IO supply. To use this functionality on the ATM2202/ATM3202, please contact Atmosic Support.
    c. To wake on threshold crossing, the user sets **pmu1**.*how2wkup*[1] and monitors either P12 or P11 via **pmu1**.*lpcomp_in_sel* with the reference voltage adjusted via **pmu11**.*lpcomp_ref*. This functionality is only valid with an external IO supply. To use this functionality on the ATM2202/ATM3202, please contact Atmosic Support.

CONFIDENTIAL

5. Powerdown (PWD pin asserted): All power domains including the PMU are completely shut off. No supplies are internally generated or maintained. This state is only valid with an external IO supply. To use this state on the ATM2202/ATM3202, please contact Atmosic Support.

2.1.10 Debug Access

There is a Serial Wire Debug (SWD) interface which provides access to the ARM M0 debug port. To enable this interface for the ATMx2x1, the input Test Mode Control pin TMC must be set to 1'b1. No other pin selection or configuration is needed. With TMC=1'b1,

- P0 = jtag_sel,
- P1 = swdclk, and
- P2 = swdio.

P0 should be set to 1'b0 and the debugger connected to P1 and P2. The SWD interface is compliant with the ARM Debug Interface Specification v5. There is a single Serial Wire Debug Port (SW-DP) and a single AHB Access Port (AHB-AP). The AHB-AP connects to the AHB backplane and has access to the entire memory mapped space. The debugger and the Cortex®-M0 share the same internal AHB master port. To prevent execution of erroneous software that blocks debug access, the Cortex®-M0 should be instructed to stay in an idle state by setting the pin P32 high and then rebooting the ATM2/ATM3. If either CSYSPWRUPREQ or CDBGPWRUPREQ in the Serial Wire Debug Port (SW-DP) registerCTRL/STAT is set, the SoC will return from a lower power mode (if in a low power mode when the bits are set) and the SoC will not go into subsequent low power modes. Refer to ARM Debug Interface Specification v5 for more information.

2.2 Security

The ATM2/ATM3 has a true random number generator (TRNG) which generates a single 32 bit random number per invocation. The number is generated when the radio is first powered up or when requested by software. Arbitrarily long random numbers can be achieved by repeatedly invoking this random number generator. The outputs of TRNG can be used to generate public and private keys which in turn can be used to generate the link layer symmetrical key. The random number generation completes in as short as 64 usec (1,024 clk_mpc clock cycles). In the unlikely event that the random number is not generated within 2 msec (32,768 clk_mpc clock cycles), an interrupt is generated to indicate the TRNG failure. In general, random number generation should be scheduled to run during periods of inactivity.

The ATM2/ATM3 also has an AES-128 hardware accelerator. This core is directly accessible by software and it is also used by the hardware to encrypt and decrypt packets when needed during Bluetooth LE operation.

If necessary a fuse within the OTP can be set to disable all slave interfaces. This would prevent keys being read out by a third party but it would also disable the SWD interface which in turn blocks future debug. Setting this disabling fuse should be used with caution. Alternatively, writes to the OTP can be permanently blocked. This mechanism would prevent the alteration of public keys but would not block the SWD interface.

2.3 Power Management Unit

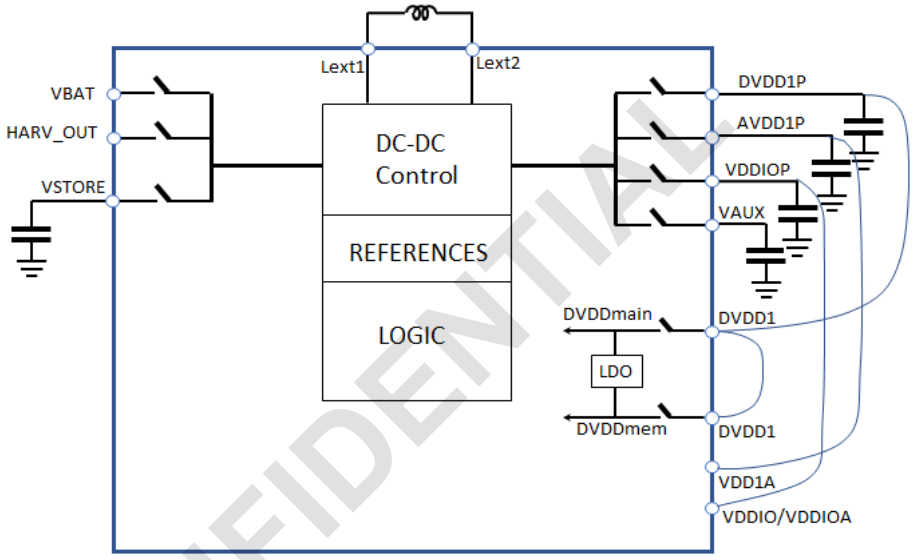
The power management unit (PMU) provides the core and IO power supplies to the ATM2/ATM3 SoC. The internal power management unit can be bypassed if the required supplies are already available on the system. A single-inductor, multi-input, multi-output switching regulator in the PMU generates the three power supplies used by the SoC: DVDD1P, AVDD1P, VDDIOP, and a fourth auxiliary supply VAUX used internally by the PMU. The PMU block diagram is shown in Figure 2.3-1. To use all of the PMU supplies, the following connections must be made on the board:

CONFIDENTIAL

- DVDD1P to DVDD1
- AVDD1P to VDD1A
- VDDIOP to VDDIO/VDDIOA

The ATM3 SoC includes an RF harvester whose output can be used by the PMU. The PMU can also be connected to other external sources of harvested energy such as photovoltaic cells, thermal electric generators (TEG), or motion. Harvested energy can extend the battery life or enable operation without a battery for some low duty-cycle applications.

Figure 2.3-1 Power Management Block Diagram



CONFIDENTIAL

The energy source can be a battery VBAT, a harvester output HARV_OUT, or an energy storage element VSTORE. The switching regulator transfers energy from the harvester output to the storage element when there is excess energy available at the harvester output.

An optional LDO can generate the RAM supply, DVDDmem, from the main internal digital supply DVDDmain, thereby allowing DVDDmem to be lower than DVDDmain. Alternatively, the LDO can be bypassed and the entire DVDD1 supply can be lowered in Retention mode for better power efficiency when using the internal PMU. In Hibernate mode, the LDO is disabled and DVDDmem is disconnected from DVDDmain, while the DVDD1 supply from the dc-dc converter can also be lowered for reduced leakage and better power efficiency.

Table 2.3-1 PMU External Pins

PIN	Description
VBAT	Battery input. Battery voltages from 1.1 V to 3.3 V can be used. Must be connected to the battery, external supply, or ground (battery-free applications). Refer to the datasheet for recommended operating conditions.
VSTORE	Connection to a large storage capacitor or rechargeable battery. Must be grounded for ATM2 or when the harvester is not used.
HARV_OUT	Connection to a capacitor of typical value 10 µF. Must be grounded for ATM2 or when the harvester is not used.

LEXT1, LEXT2	Terminals between which the switching regulator inductor is connected.
DVDD1P, AVDD1P VDDIOP	DVDD1P and AVDD1P are PMU generated digital and analog core supply outputs. VDDIOP is PMU generated 1.8 V IO supply output.
VAUX	Auxiliary supply output of typical value 3.2 V, used internally by the PMU.
DVDD1, VDD1A	Power supply input for digital and analog core circuits.
VDDIO, VDDIOA	Power supply input for digital and analog IO circuits.

The PMU provides multiple brownout interrupts to enable more reliable operation, especially when using energy harvesting.

PMU configurations

The PMU must be configured properly to ensure correct operation. Fields in Table 2.3-2 must be programmed by the user depending on the desired board configuration:

Table 2.3 -2 PMU Configuration Fields

Register Field	Description
otp_batt_type	0 (default) for VBAT <= 1.8 V 1 for VBAT > 1.8 V 2 for no external source on VBAT 3 for rechargeable battery > 1.8 V on VBAT
otp_harv_type (ATM3)	0 (default) for energy sources other than RF 1 for RF harvesting
otp_disable_vddIO	0 (default) Set to 1 to disable internal VDDIO generation. VDDIO must be provided externally.
otp_brownout_thr	VBAT brownout threshold level 0 (default) 0.95 V for otp_batt_type = 0, 1.9 V for otp_batt_type = 1 or 3 2 1.00 V for otp_batt_type = 0, 2.0 V for otp_batt_type = 1 or 3 4 1.05 V for otp_batt_type = 0, 2.1 V for otp_batt_type = 1 or 3 6 1.10 V for otp_batt_type = 0, 2.2 V for otp_batt_type = 1 or 3

Table 2.3-3 shows PMU configurations supported by ATM2 and ATM3.

Table 2.3 -3 PMU Supported Configurations

Battery Type and IO Configuration	External IO Supply	Internal IO Supply
Regular battery <= 1.8 V on VBAT		ATM2, ATM3
Regular battery > 1.8 V on VBAT	ATM2, ATM3	ATM2, ATM3
No VBAT	ATM3*	ATM3
Rechargeable battery > 1.8 V on VBAT	ATM3	ATM3

*To use this functionality on the ATM3, please contact Atmosic Support

CONFIDENTIAL

When using the SDK, the PMU_CFG or CFLAGS makefile argument will program the PMU configuration fields in software or in the hardware OTP depending on the makefile target. The supported PMU modes of operation break down to the following categories, and will be discussed in detail.

- Battery options
- Internal or external IO supply
- System brownout threshold
- Harvesting type (ATM3 only)
- VSTORE maximum threshold (ATM3 only)

Battery options

- *otp_batt_type* = 0 for VBAT <= 1.8 V,
  - 1 for VBAT > 1.8 V,
  - 2 for batteryless operation,
  - 3 for rechargeable battery.

  VBAT cannot be accessed once this setting is set to 2
- For regular battery with VBAT <=1.8 V, add in makefile: `PMU_CFG = VBAT_LE_1p8V`
- For regular battery with VBAT > 1.8 V, depending on internal/external IO configuration (discussed below), add in makefile: `PMU_CFG = VBAT_GT_1p8V_VDDIO_INT` or `VBAT_GT_1p8V_VDDIO_EXT`
- For batteryless operation, add in makefile: `PMU_CFG = NO_VBAT`. VBAT cannot be accessed once this setting is applied.
- For rechargeable battery, add CFLAGS in makefile: `-DCFG_RECHBATT`
- If not set, the default setting is 0 indicating VBAT<=1.8 V. If this setting is not set properly, the brownout levels (discussed below) might be incorrectly configured.

CONFIDENTIAL

Internal or external IO supply

For applications that cannot support a fixed 1.8 V IO supply, use PWD, or use external wakeup from SoC Off, option (a) should be used for external IO supply. For applications that can use a fixed 1.8 V IO supply for better power consumption or VBAT <= 1.8V, option (b) should be used for internal IO supply.

(a) One external power supply or battery with external IO supply:

- Connect VBAT to VDDIO/VDDIOA
- Connect VAUX to VDDIOP
- *otp_disable_vddIO* = 1
- For regular battery with VBAT > 1.8 V or rechargeable battery, add in makefile: `PMU_CFG = VBAT_GT_1p8V_VDDIO_EXT`

(b) One external power supply or battery with internally generated IO supply:

- VDDIO/VDDIOA = 1.8 V
- *otp_disable_vddIO* = 0
- This is the default setting.

System brownout threshold

Please refer to the PSEQ section for more details on brownout functionalities.

- To program the threshold for systems with batteries, add CFLAGS in makefile: `-DBRWNOUT_THR=<X>`. The available settings are shown in Table 2.3-4.

Table 2.3-4 shows the `-DBRWNOUT_THR=<x>` setting for specific VBAT threshold level setting.

Table 2.3-4 VBAT Brownout Threshold Level and CFLAGS Setting

VBAT brownout threshold level For otp_batt_type = 0	VBAT brownout threshold level For otp_batt_type = 1 or 3	CFLAGS setting
0.95 V	1.9 V	DBRWNOUT_THR = 0 (default)
1.00 V	2.0 V	DBRWNOUT_THR = 2
1.05 V	2.1 V	DBRWNOUT_THR = 4
1.10 V	2.2 V	DBRWNOUT_THR = 6

- For batteryless setting, the brownout threshold is automatically set at VSTORE = 1.6 V, with a hysteresis level of VSTORE = 1.75 V.
- The brownout functionality is enabled by default and is recommended for systems with batteries. Please note that the battery type setting needs to be configured properly for the brownout threshold setting to take effect. To disable this functionality, add in makefile: `NOBROWNOUT = 1`
- System brownout functionality is only supported for flash applications. Please contact Atmosic to implement this functionality for OTP applications.

CONFIDENTIAL

Harvesting type (ATM3 only)

(a) RF Energy harvesting operation:

- *otp_harv_type* = 1 for RF harvesting
- Add CFLAGS in makefile: `-DCFG_RF_HARV`

(b) Photovoltaic, Thermal, Mechanical Energy harvesting operation:

- *otp_harv_type* = 0 for energy sources other than RF
- Add 2 CFLAGS in makefile: `-DCFG_NONRF_HARV -DCFG_NONRF_VRANGE_1pX_1pX`

Table 2.3-5 shows the -DCFG_NONRF_VRANGE_xxx_xxx setting for specific HARV_OUT desired voltage range:

Table 2.3-5 Voltage Range and CFLAGS Setting

HARV_OUT desired voltage range	CFLAGS setting
1.1 V - 1.4 V	Please contact Atmosic to implement this setting
1.2 V - 1.5 V	Please contact Atmosic to implement this setting
1.3 V - 1.6 V	DCFG_NONRF_VRANGE_1p3_1p6
1.4 V - 1.7 V	DCFG_NONRF_VRANGE_1p4_1p7
1.5 V - 1.8 V	DCFG_NONRF_VRANGE_1p5_1p8
1.6 V - 1.9 V	DCFG_NONRF_VRANGE_1p6_1p9 (default)

VSTORE maximum threshold (ATM3 only)

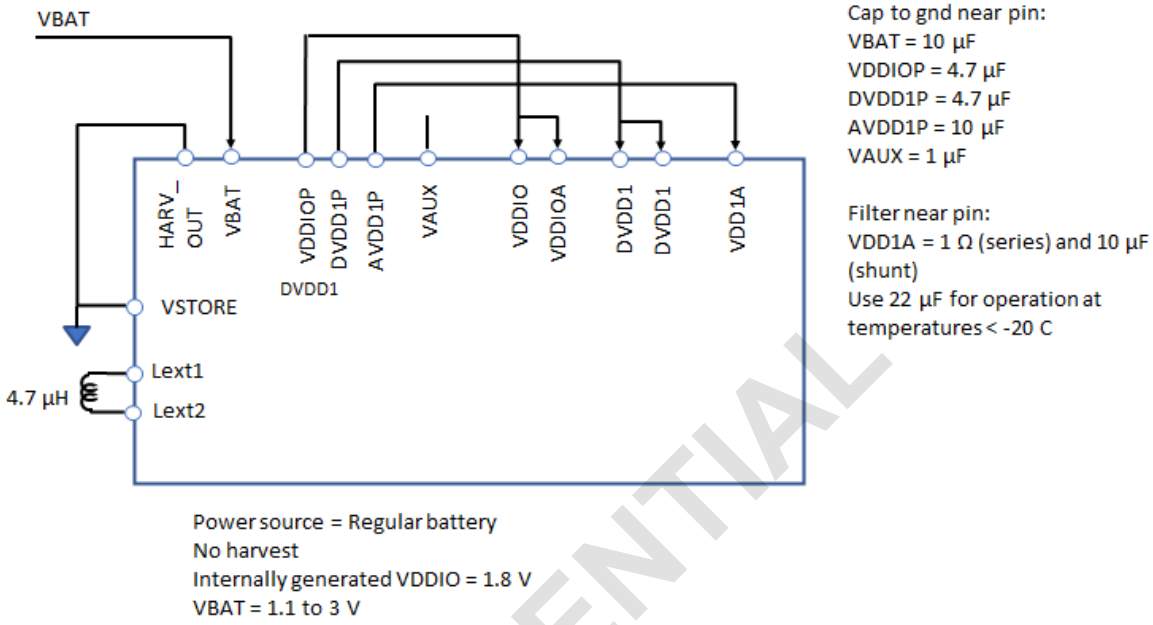
- The storage element connected on VSTORE can be limited to a max threshold of 3V or 3.3 V. Add in makefile: `-DVSTORE_MAX_EQ_3p0V` or `-DVSTORE_MAX_EQ_3p3V`. The default setting is 3.3 V.
- For a system with rechargeable batteries connected at VSTORE, this setting serves as an upper voltage protection of the battery.

Block diagrams for all supported PMU configurations:

(1) Regular battery <=1.8 V or >1.8 V on VBAT, with internal IO supply, and no harvesting capability. See Figure 2.3-2.

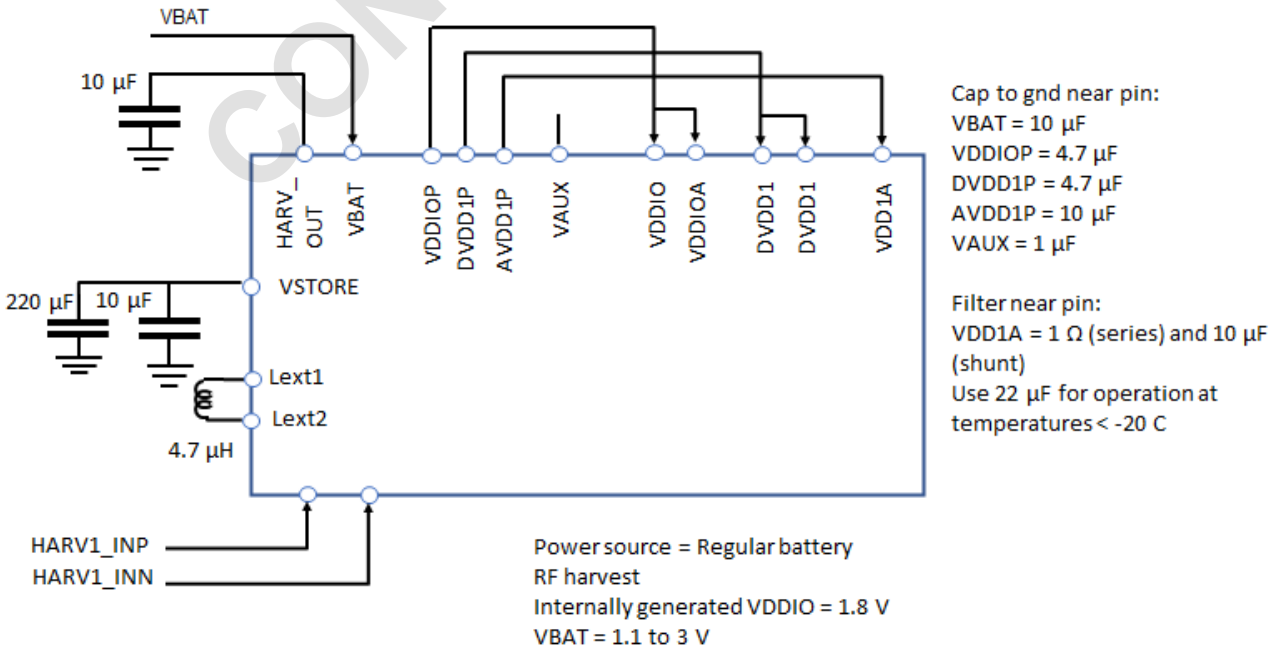
CONFIDENTIAL

Figure 2.3-2 Regular Battery, Internal IO Supply, No Harvesting Capability



(2) Regular battery <=1.8V or >1.8V on VBAT, with internal IO supply, and RF harvesting source

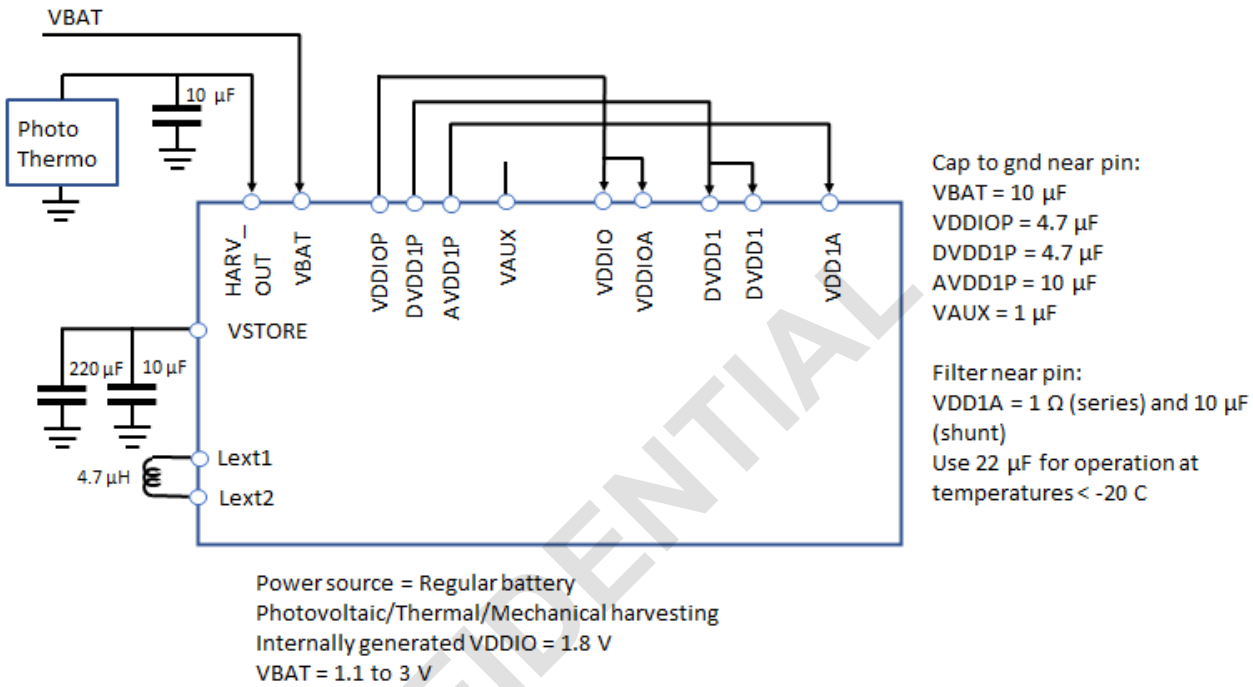
Figure 2.3-3 Regular Battery, Internal IO Supply, RF Harvesting Source



(3) Regular battery <=1.8V or >1.8V on VBAT, with internal IO supply, and non-RF harvesting source

CONFIDENTIAL

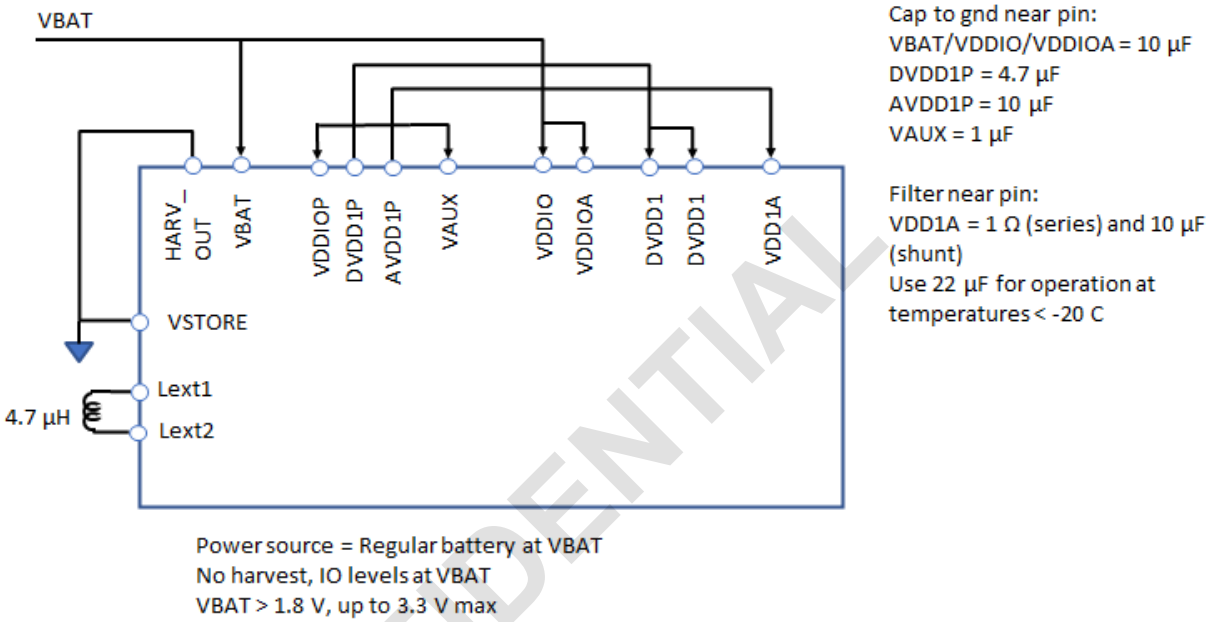
Figure 2.3-4 Regular Battery, Internal IO Supply, Non-RF Harvesting Source



CONFIDENTIAL

(4) Regular battery >1.8V on VBAT, with external IO supply, and no harvesting capability

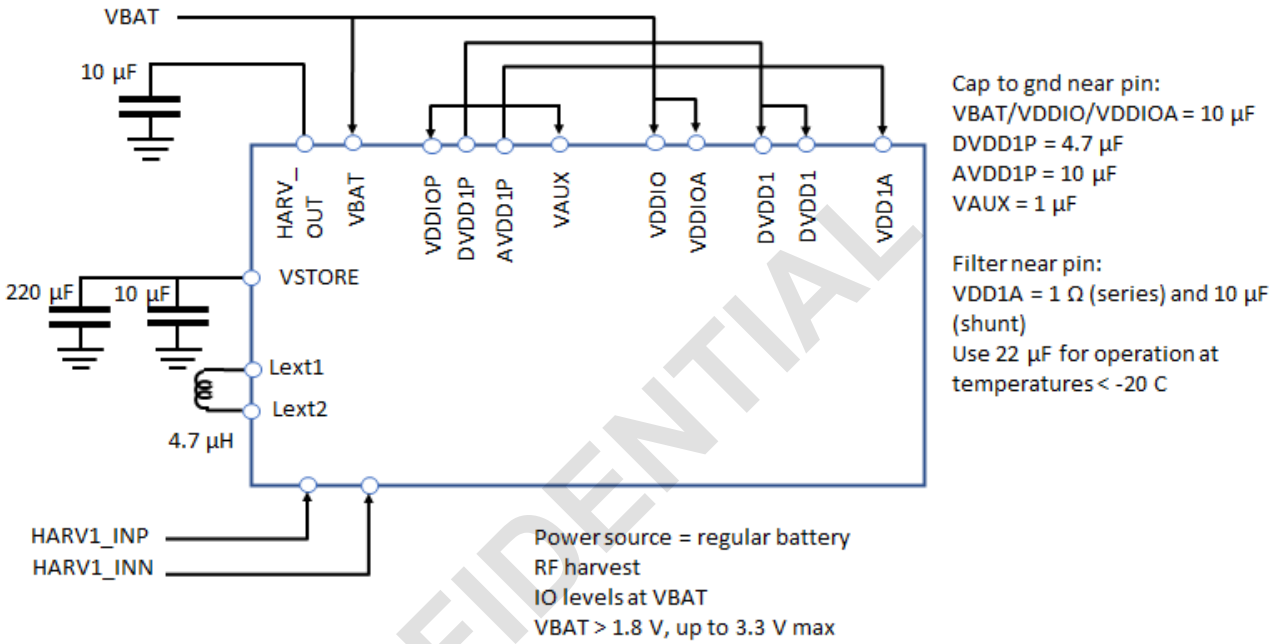
Figure 2.3-5 Regular Battery, External IO Supply, No Harvesting Capability



CONFIDENTIAL

(5) Regular battery > 1.8 V on VBAT, with external IO supply, and RF harvesting source

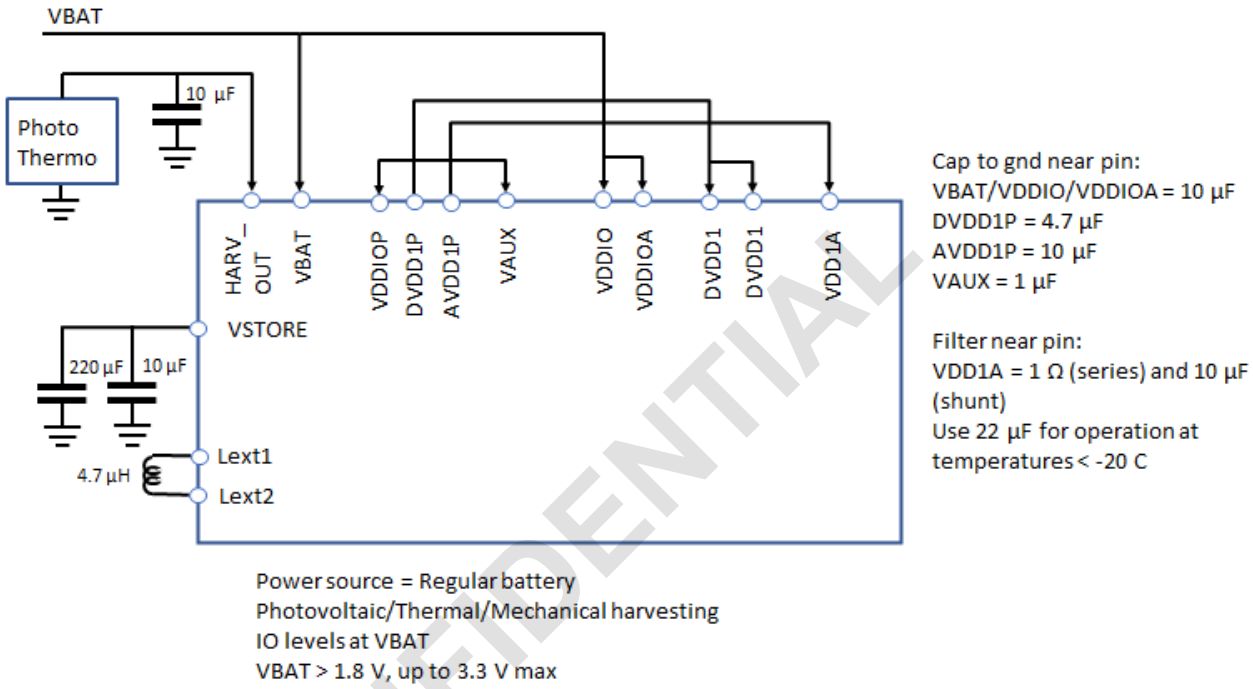
Figure 2.3-6 Regular Battery, External IO Supply, RF Harvesting Source



CONFIDENTIAL

(6) Regular battery > 1.8 V on VBAT, with external IO supply, and non-RF harvesting source

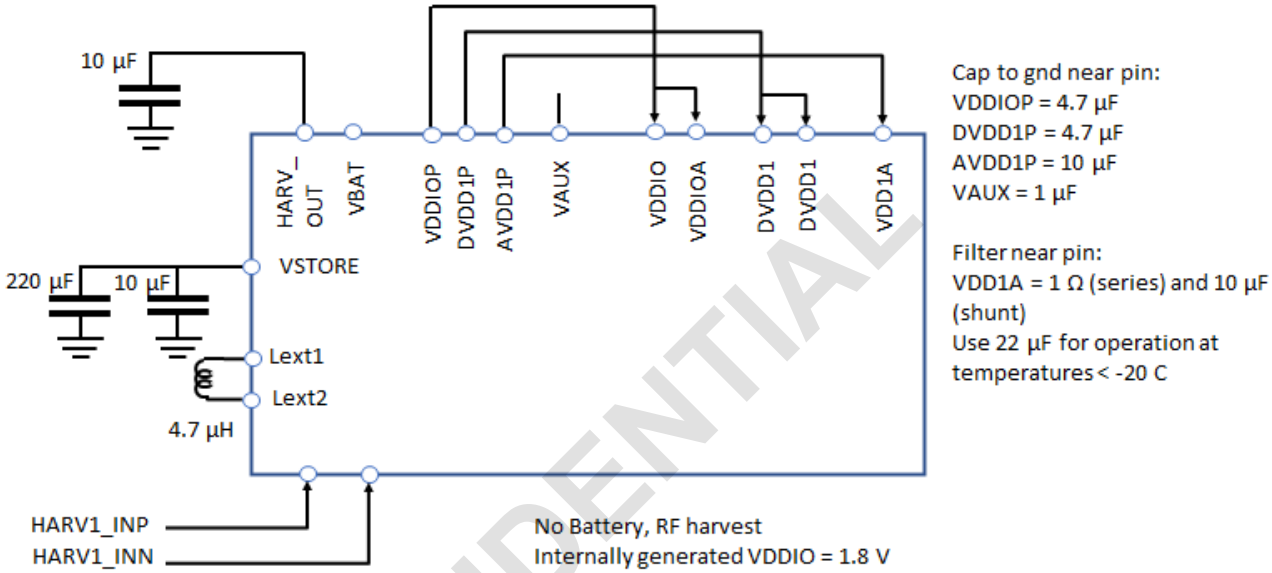
Figure 2.3-7 Regular Battery, External IO Supply, Non-RF Harvesting Source



CONFIDENTIAL

(7) No battery on VBAT, with internal IO supply, and RF harvesting source

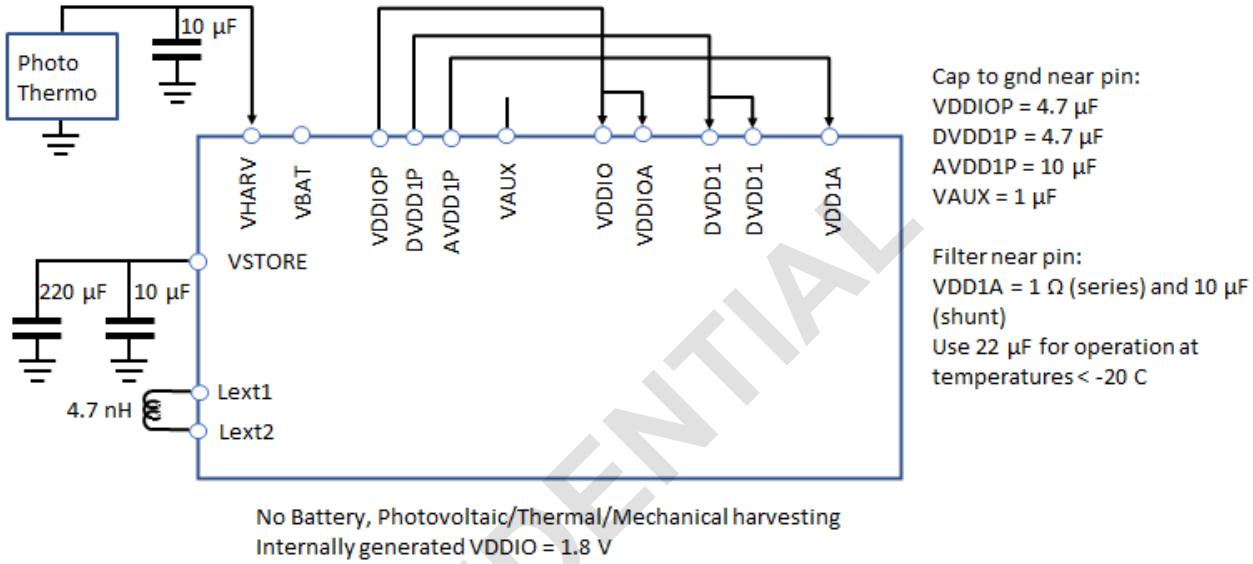
Figure 2.3-8 No Battery, Internal IO Supply, RF Harvesting Source



CONFIDENTIAL

(8) No battery on VBAT, with internal IO supply, and non-RF harvesting source

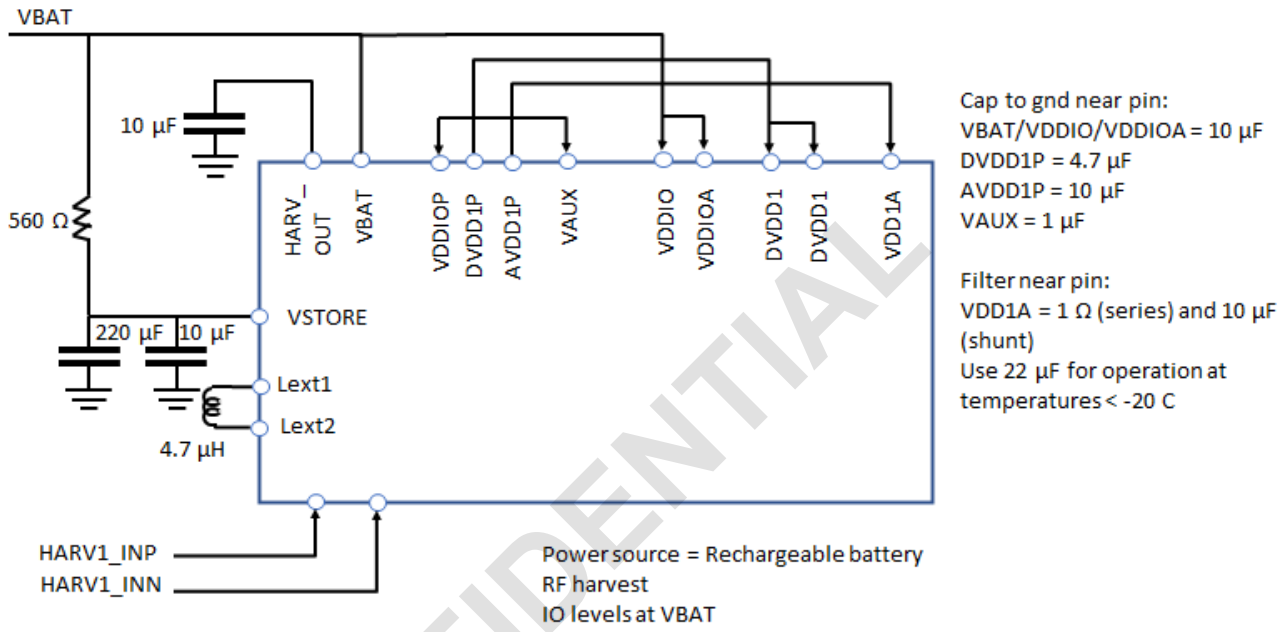
Figure 2.3-9 No Battery, Internal IO Supply, non-RF Harvesting Source



CONFIDENTIAL

(9) Rechargeable battery on VBAT, with external IO supply, and RF harvesting source

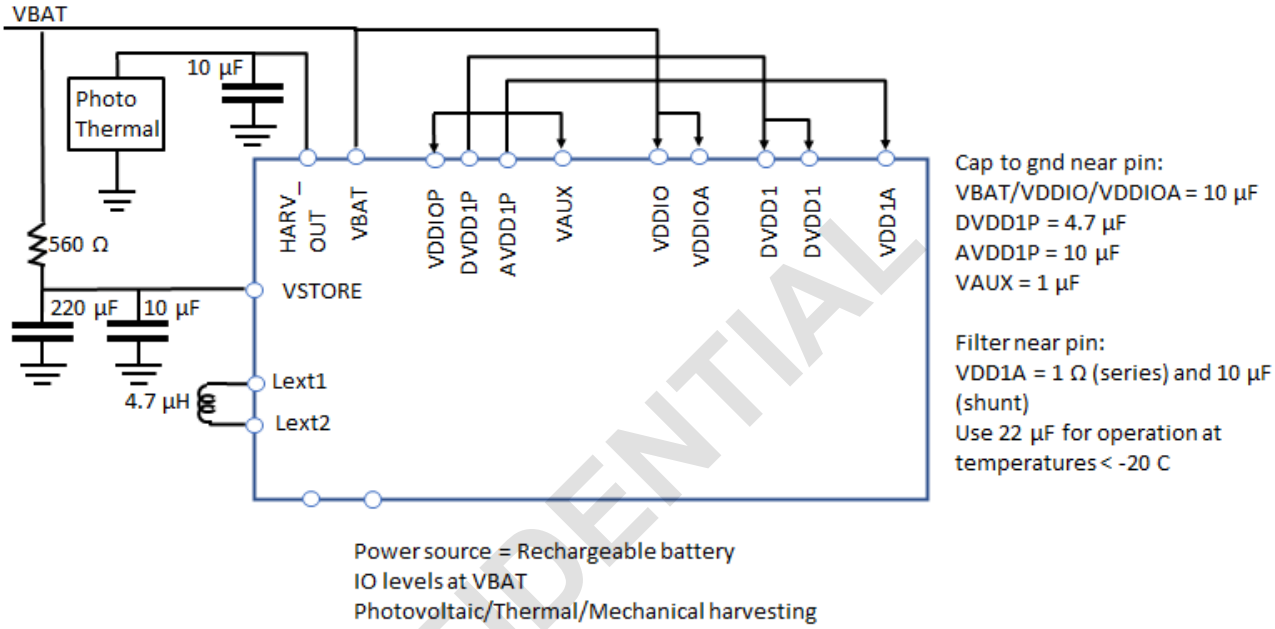
Figure 2.3-10 Rechargeable Battery, External IO Supply, RF Harvesting Source



CONFIDENTIAL

(10) Rechargeable battery on VBAT, with external IO supply, and non-RF harvesting source

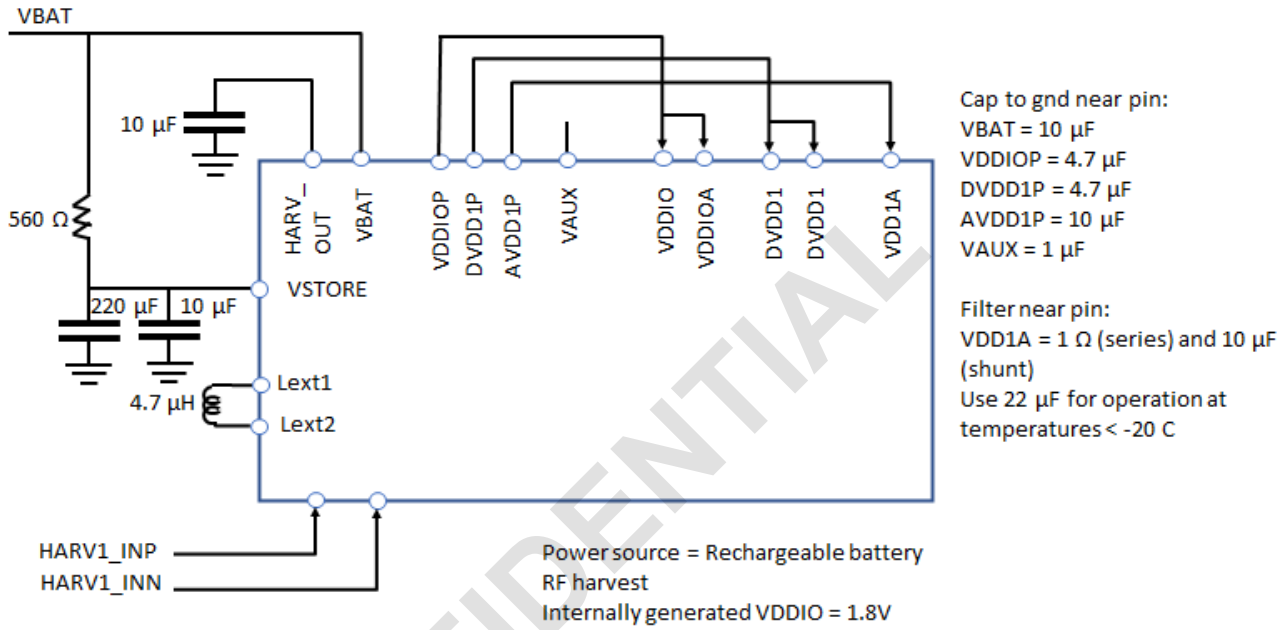
Figure 2.3-11 Rechargeable Battery, External IO Supply, Non-RF Harvesting Source



CONFIDENTIAL

(11) Rechargeable battery on VBAT, with internal IO supply, and RF harvesting source

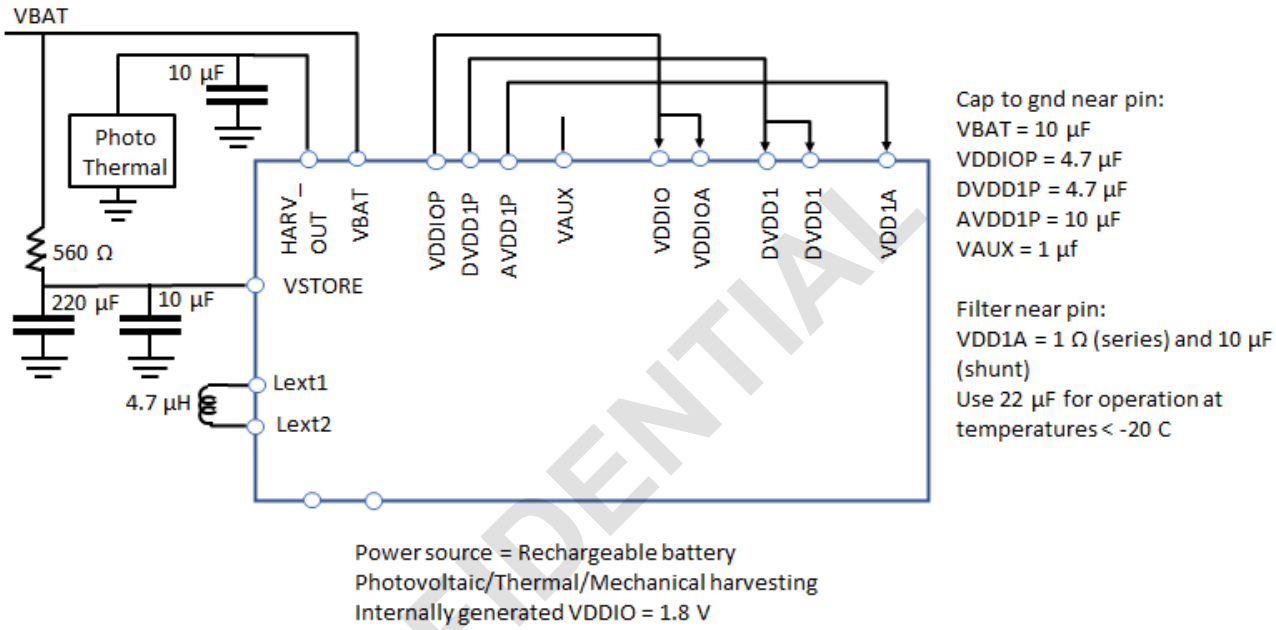
Figure 2.3-12 Rechargeable Battery, Internal IO Supply, RF Harvesting Source



CONFIDENTIAL

(12) Rechargeable battery on VBAT, with internal IO supply, and non-RF harvesting source

Figure 2.3-13 Rechargeable Battery, Internal IO Supply, Non-RF Harvesting Source



CONFIDENTIAL

2.4 Sensor Hub

The sensor hub is a hardware module that, when programmed, can read data from external sensors and write to flash on the quad SPI interface. It can do this function even with all other power domains are powered down, including the CPU power domain. The sensor hub can trigger a wake up interrupt if any of the read data crosses preprogrammed thresholds. Refer to the ATM2/3 Sensor Hub White Paper for additional information.

The sensor hub core, running on a 32.768 kHz CLK_LPC clock, polls data from sensors and transfers them to the flash. Connectivity to sensors is provided by two ports, with each port configurable as either SPI or I2C protocol. At the system level, the user connects the sensors to the appropriate pins reserved for the sensor hub and programs the pin mux accordingly.

- Port0: The first configuration for port 0 is in **port0_setup** register which specifies: the type of protocol (*spi_n_i2c*), the number of sensors connected to this port (*num_sensors*), off/on state of the port (*port_en*) and whether the sensor data are to be compared with pre-programmed threshold quantities (*trig_en*).

  If the protocol is SPI, only one sensor is expected and the user must program the *sensor_opcode* in the **port0_spi_eng0_ctrl** register and the number of bytes *num_bytes* in the **port0_sensor0_setup** register to be read from the sensor. Note that an extra byte will be read from the sensor and written to flash, so it should be discarded by the application.

  If the protocol is I2C, up to 8 sensors can be connected to port0 as specified in **port0_setup**.*num_sensors*. For each sensor, the corresponding **port0_sensor**x**_setup** register should be programmed with the number of bytes to be read from the sensor minus one. Furthermore, the user must program the control registers for I2C packets, each of which corresponds to one of 4 stages making up for one sequence of I2C data transfer from a sensor. These 4 stages are <sensor id>, <pointer address within the sensor>, <sensor_id> <sensor data> ... <sensor data>, and the last stage (sensor data) can be multiple packets equal to the number of bytes. The ID and address stages are programmed in the corresponding **port0_i2c_idw_**x, **port0_i2c_addr_**x, and **port0_i2c_idr_**x registers. The data stage is specified in two registers which are common for all sensors: **port0_i2c_dat_ctrl** for the first and intermediate bytes and **port0_i2c_dat_last_ctrl** for the last byte in each read sequence.

- Port1: The first configuration for port 1 is in **port1_setup** which specifies: the type of protocol (*spi_n_i2c*), the number of sensors connected to this port (*num_sensors*), off/on state of the port (*port_en*) and whether the sensor data are to be compared with pre-programmed threshold quantities (*trig_en*).

  If the protocol is SPI, only one sensor is expected and the user must program the *sensor_opcode* in the **port1_spi_eng0_ctrl** register, and the number of bytes *num_bytes* in the **port1_sensor0_setup** register to be read from the sensor. Note that an extra byte will be read from the sensor and written to flash, so it should be discarded by the application.

  If the protocol is I2C, only one sensor can be connected to port1 and the **port1_sensor0_setup** register should be programmed with the number of bytes to be read from the sensor minus one. Furthermore, the user must program the control registers for I2C packets, each of which corresponds to one of 4 stages making up for one sequence of I2C data transfer from a sensor. These 4 stages are <sensor id>, <pointer address within the sensor>, <sensor_id> <sensor data> ... <sensor data>, whereby the last stage (sensor data) can be multiple packets equal to programmed number of bytes. The ID and address stages are programmed in the **port1_i2c_idw_0**, **port1_i2c_addr_0**, and **port1_i2c_idr_0** registers. The data stage is specified in two registers: **port1_i2c_dat_ctrl** for the first and intermediate bytes and **port1_i2c_dat_last_ctrl** for the last byte in each read sequence.

2.4.1 Event schedule

The sensor hub main FSM is triggered by the PSEQ. Once triggered, the FSM reads the data from the sensors and writes to the flash repeatedly until the PSEQ requests for a stop. The cycle duration is set by sensor hub **setup1** register which shall be programmed longer than the time it takes to read all sensors' data and write to flash.

CONFIDENTIAL

In each cycle (controlled by **setup1**), sensors on port0 and port1 are read simultaneously. Starting with sensor 0, the FSM reads the data from both ports and writes port 0's data and then port 1's data to the flash. For subsequent sensors, the FSM only receives the data from port 0. Note that, the write phase is broken up to 2 actual flash writes if the bulk of data bytes span page boundaries.

2.4.2 Data comparison

Port 0 is equipped with one alarm for sensor 0 when only one sensor is connected. Port 1 is equipped with one alarm for the single sensor that can be attached to it. In this section, the alarm on port 0 is used as the reference for description. The alarm treats the stream of bytes from sensor 0 in terms of distinct quantities, each of which is made up of a number of contiguous bytes. The user can define up to 4 quantities which are specified in the **port0_alm0_quan_ctrl** register by their size and starting positions in the data stream (up to 32 bytes). The quantities cannot overlap, and each quantity can be at most 3 bytes, so the combined number of bytes is at most 12 bytes.

- If the quantities have their most significant bytes first, the user must be programmed (**port0_alm0_quan_ctrl**.*lsb_first*) to 0.

- **port0_alm0_quan_ctrl**.*quan_dat_is_signed* specifies whether the quantities are signed numbers.

- Next, set the floor and ceiling reference quantities with which the sensor's quantities are compared. 12 bytes are allocated for the floor and the ceiling. For the ceiling reference, the 12 bytes, *byte_max_00*.... *byte_max_11*, are spread across registers **port0_alm0_thrhld_max_0**, **port0_alm0_thrhld_max_1** and **port0_alm0_thrhld_max_2**. For the floor reference, the 12 bytes, *byte_min_00*.... *byte_min_11*, are spread across registers **port0_alm0_thrhld_min_0**, **port0_alm0_thrhld_min_1** and **port0_alm0_thrhld_min_2**.

- For each reference, the hardware packs the bytes together into quantities in similar fashion as it does to the sensor's quantities. Each reference quantity is determined by the sensor quantity size. For instance, if *quan0_size* = 1, *quan1_size* = 2 , *quan2_size* = 2 and *quan3_size* = 0, then the groupings are as follows:
    - {byte_min_00, byte_min_01}, {byte_min_02, byte_min_03, byte_min_04}, {byte_min_05, byte_min_06, byte_min_07}, {byte_min_08}
    - {byte_max_00, byte_max_01}, {byte_max_02, byte_max_03, byte_max_04}, {byte_max_05, byte_max_06, byte_max_07}, {byte_max_08}

- All 4 sensor quantities are then compared to their respective references and have their out of range status captured in **port0_status**.*out_of_range_stat*. The total number of possible values of the 4-bit *out_of_range_stat* is 15 (disregarding 0). The user can choose any value to trigger the CPU by programming the desired value in **port0_alm0_trig_ctrl**.

- Last, each port has its trigger enable in **port**x**_setup**.*trig_en*

2.4.3 Flash control

The sensor data is written to the flash. The user must specify the following items for the flash operation:

- *Opcodes* in **flash_ctrl0** register for write enable, page program, power down and release from power down. Set **flash_ctrl3**.*is_macronix* and **flash_ctrl3**.*serialize_pp_address* depending on the flash vendor.

- Start address in **flash_ctrl1** register. This flash address is where the first data byte is written to.

- Max addr in **flash_ctrl2** register. This is the last address for sensor data. The sensor hub FSM will wake up the CPU when this address is reached. The number of leftover data bytes are stored in *lost_write_cnt* in **flash_status1**

CONFIDENTIAL

register and **flash_status0**.*wr_saturated* is set by the hardware. These status registers can be cleared by writing ones to **flash_ctrl3**.*clear_lost_write_cnt* and **flash_ctrl3**.*clear_flash_overflow*, respectively.

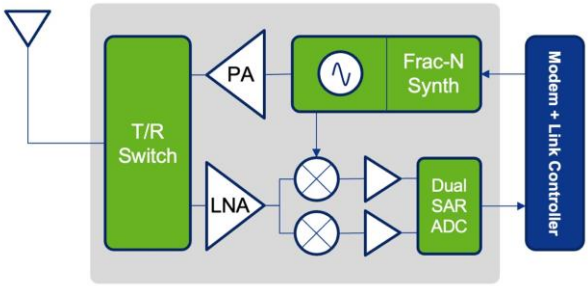
- Interface mode in **flash_ctrl3**.*qspi_mode* determines whether the interface is single SPI or quad SPI.
- Page program delay time in **flash_ctrl4**.*pp_delay.* After a page program opcode, the flash takes some time to complete the write process.
- Power cycling mode: This mode is controlled by setting **flash_ctrl3**.*power_cycle_en* to one. In this mode, the flash chip is put into a low power state after each write and recalled to active state at the start of the sensor read phase. To account for the flash recovery time from the low power state, the user can program for a delay time in **flash_ctrl4**.*pd_delay*. The write phase only starts if both the read phase and delay **flash_ctrl4**.*pd_delay* are completed. For Winbond and Micron, set **flash_ctrl3**.*rpd_has_clock* = 1.

2.5 Bluetooth LE Radio

The block diagram of the low-power radio is shown in Figure 2.5-1. The radio supports Bluetooth 5.0 including 1 Mbps basic PHY, 2 Mbps high-speed PHY, 500 kbps and 125 kbps PHY. Refer to the Bluetooth LE Coded PHY white paper for more information.

The basic 1 Mbps PHY is compatible with Bluetooth 4.0, while the 2 Mbps rate provides 2X speed and the long range rates provide up to 4X range. The transmit path uses digital direct frequency modulation of a fractional-N synthesizer to create a constant amplitude GFSK signal that is amplified by a power amplifier to provide the desired RF output level. On the receive path, an incoming RF signal is first amplified by an LNA before downconversion to baseband and digitized by two successive-approximation analog-to-digital converters. The digitized signal is sent to the Modem for further digital processing. The radio architecture is optimized for burst data transmission using Frequency-Hopping Spread Spectrum (FHSS) with 40 channels with 2 MHz spacing (3 advertising channels/37 data channels). Only a single RF Input/Output pin is needed, thereby simplifying board-level design.

Figure 2.5-1 Bluetooth LE Radio Block Diagram



2.5.1 Link Controller

The Bluetooth 5 link controller (LC) and host stack provides interface between the microcontroller (MCU), modem (MDM) and exchange memory (EM), allowing the MCU to access through AHB bus to the control registers and exchange memory. During transmit, software writes the packet payload and control structures into the exchange memory. The link controller

CONFIDENTIAL

serializes the data into a bit stream to the modem. During receive, the operation is reversed. Received data from the modem is processed and stored in the exchange memory to be read by software.

2.5.2 LC Module Descriptions

- Interrupt Control handles interrupt to MCU.
- Wi-Fi Coexistence follows the 3-wire packet traffic arbitration protocol to ensure minimal interference between Wi-Fi and Bluetooth LE radio.
- AHB interface translates AMBA AHB bus slave signals to control and data access to registers and exchange memory.
- Control Reg contains control and status registers.
- Timer interfaces to the lower power control module which resides in always on power domain and can wake up Bluetooth LE LC on timeout or wakeup event detection.
- Crypto Proc encrypts and decrypts payload based on AES-CCM.
- Bit Proc performs bit stream operations like whitening/de-whitening, CRC generation/check, preamble generation.
- Core
  - Mem Control arbitrates memory access requests from various masters
  - Event Schloss starts an event by pref-etching entries in exchange table in exchange memory.
  - Event Control reads and writes control structures and descriptors from and to exchange memory.
  - Pkt Control controls the sequence of operations performed by the bit processor.
  - Srch Eng parses white list or periodic advertiser list and ADI list stored in exchange memory.
  - RAL Srch Eng is in charge of private address resolution and generation, managing RAL structure stored in exchange memory.
  - Freq Control calculates the frequency to be used for transmission and reception.

Tx Data Flow

1. SW writes transmit packet payload and control structures into the exchange memory.
2. Crypto processor reads and encrypts payload.
3. Packet controller reads encrypted payload from crypto processor or unencrypted payload from exchange memory and passes to bit processor.
4. Bit processor generates PPDU by whitening the payload, prepending preamble and header, post pending CRC and FEC encoding the PPDU.
5. Serialized bit stream is passed to the radio controller.
6. Radio controller passes the bit stream to radio for transmission.

Rx Data Flow

1. MDM correlates preamble and access address for packet detection and FEC decodes and demaps long range encoded packets.
2. Radio controller passes the received bit stream to the bit processor.
3. Bit processor whitens the received bit stream and checks the CRC.
4. Packet controller passes the encrypted payload to the crypto processor or unencrypted payload to exchange memory.
5. Crypto processor decrypted payload and writes to exchange memory.
6. SW reads MPDU.

2.5.3 Modem

The modem (MDM), along with the radio, link controller and software stack, forms a highly efficient Bluetooth 5 solution. Transmission and reception at rates of 2 Mbps, 1 Mbps, 500 kbps and 125 kbps are supported.

CONFIDENTIAL

2.5.3.1 Receiver

The modem and radio are enabled prior to expected packet reception as determined by the link controller. Baseline power measurements of the incoming signal strength are made which serve as a reference for future measurements. Packet detection is accomplished through a combination of power measurements, signal saturation, and correlation with the expected Bluetooth LE preamble. Analog receive gain is adjusted based on the incoming signal strength. Initial estimation of carrier frequency offset and timing synchronization is performed early in the packet, followed by continuous tracking throughout. Correlation with the expected access address is performed and is indicated to the link controller if found. A Received Signal Strength Indicator (RSSI) is also sent to the link controller if the access address is found.

The modem and radio receiver are turned off by the link controller at the end of a completed packet or through various error conditions. The modem may initiate a restart of the packet detection process if the access address is not found, or if synchronization is not achieved.

Channel information is provided directly from the link controller to the radio without involvement of the modem.

The default modem and link controller register settings are optimized for non-scanning operation. For applications that require scanning, the hw_cfg_optimize_scan_params and hw_cfg_restore_scan_params SDK functions in platform/atm2/ATM2xE3c/driver/hw_cfg/hw_cfg.c should be called when scanning starts and ends, respectively. For example, the SDK calls those functions inside appm_start_scanning and appm_stop_scanning in platform/atm2/ATM2xE3c/app/src/app.c. For applications that require maintaining a connection while scanning, the functions should not be used.

2.5.3.2 Transmitter

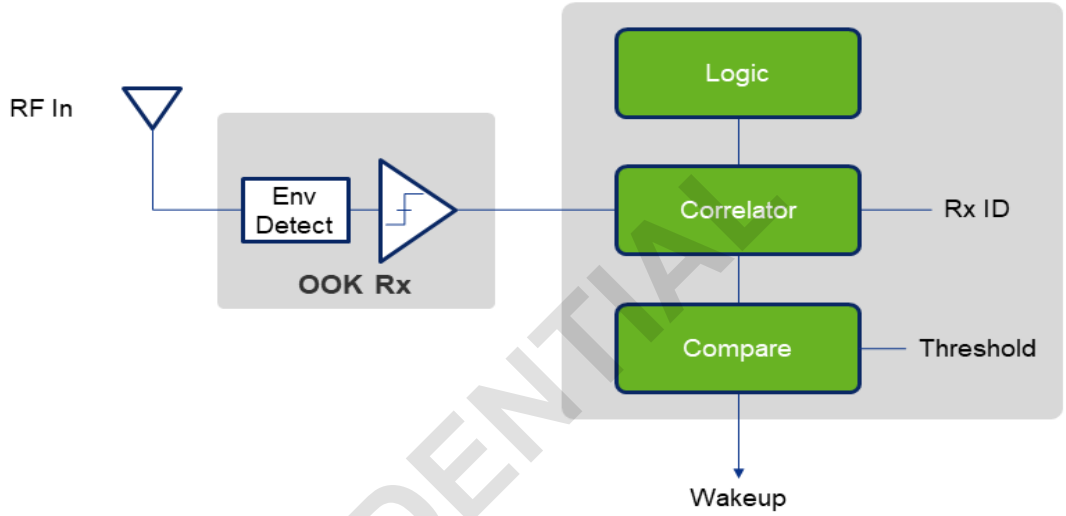
There is minimal involvement of the modem in the transmit process. The link controller provides the symbol bit stream, which is then shaped consistent with the GFSK requirements to provide a frequency deviation for the radio.

Channel information and target transmit power are provided directly from the link controller to the radio.

CONFIDENTIAL

2.6 Wakeup Receiver

Figure 2.6-1 Wakeup Receiver Block Diagram



CONFIDENTIAL

The use of a wakeup receiver allows a system to be in sleep mode while waiting for incoming RF activities. In this SoC, the wakeup receiver is designed to decode an incoming RF paging or wakeup signal with very low power consumption. This dedicated low-power wakeup receiver continuously monitors the incoming RF signal for a predefined paging signal. This continuous Rx mode is based on an OOK radio, which has ultra-low power consumption. The wakeup receiver is intended for short range and short latency applications. Latency of the wakeup receiver is typically in the order of 20 ms to 1 s, depending on the length of the Rx ID code used to identify the target device.

Refer to the ATM2/3 Wakeup Receiver White Paper for more details.

2.7 Pin Multiplexing

In the ATM3221/ATM2221 SoC, all 34 programmable IO pins, P0 - P33, support multiple functional signals. The ATM3201/ATM2201 SoC supports 17 programmable IO pins; pins that are not supported are highlighted in gray in Table 2.7-1. The ATM3202/ATM2202 SoC supports 12 programmable IO pins; pins that are not supported are highlighted in gray and blue in Table 2.7-1. The ATM3231/ATM2231 SoC supports 33 programmable I/O pins, P0 - P26, P28 - P33; P27 is not supported and is identified via footnote 3 in Table 2.7-1, Table 2.7-2, Table 2.7-3, Table 2.7-4.

Refer to the Pin Assignment and Pinmux Programming White Paper for further guidance. Using the Atmosic Pinmux Tool in the SDK is recommended for selecting digital IO functionality.

Table 2.7-1 Multiplexed Pin Function

Physical Pin	Function							
P0	GPIO_0	KSI_19	KSO_19	KSO_8		QDZ_1		
P1	GPIO_1	KSI_18	KSO_18	KSO_9		QDZ_0		
P2	GPIO_2	KSI_17	KSO_17	KSO_10	KSO_8	QDY_1		
P3	GPIO_3	KSI_16	KSO_16	KSO_11	KSO_9	QDY_0	UART1_RTS	I2C0_SCK
P4	GPIO_4	KSI_15	KSO_15	KSO_12	KSO_10		UART1_RX	I2C0_SDA
P5	GPIO_5	KSI_14	KSO_14	KSO_13	KSO_11		UART1_TX	PWM_0
P6	GPIO_6	KSI_13	KSO_13	KSO_14	KSO_12		UART1_CTS	SPI1_CLK
P7	GPIO_7	KSI_12	KSO_13	KSO_15	KSO_13		PWM_0	SPI1_CS
P8	GPIO_8	KSI_11	KSO_11	KSO_16	KSO_14		PWM_1	SPI1_MOSI
P9	GPIO_9	KSI_10	KSO_10	KSO_17	KSO_15		PWM_2	SP1I_MISO
P10	GPIO_10	KSI_9	KSO_9	KSO_18	KSO_16	UART0_RX	PWM_1	SPI0_CLK
P11	GPIO_11	KSI_8	KSO_8	KSO_19	KSO_17	UART0_TX	PWM_2	SPI0_CS
P12	GPIO_12	KSI_7			KSO_18	QDZ1	SPI0_MOSI	I2C1_SCK
P13	GPIO_13	KSI_6			KSO_19	QDZ0	SPI0_MISO	I2C1_SDA
P14	QSPI_CLK							
P15	QSPI_CS							
P16	QSPI_D0							
P17	QSPI_D1			QPSI_CLK				
P18	QSPI_D2	KSI_5	GPIO_14					
P19	QSPI_D3	KSI_4	GPIO_15	QPSI_CS				

CONFIDENTIAL

P20	GPIO_16	KSI_3		QPSI_D0			PWM_3	SPI0_CLK
P21	GPIO_17	KSI_2	UART0_CTS				PWM_4	SPI0_CS
P22	GPIO_18	KSI_1		QPSI_D1		UART0_RX	PWM_5	SPI0_MOSI
P23	GPIO_19	KSI_0		QPSI_D2		UART0_TX	PWM_6	SPI0_MISO
P24	GPIO_20	KSO_7	UART0_RTS	QPSI_D3	PDM_CLK	QDY_1	PWM_3	SPI1_CLK
P25	GPIO_21	KSO_6	UART0_RX		PDM_IN	QDY_0	PWM_4	SPI1_CS
P26	GPIO_22	KSO_5				DQX_1	PWM_5	SPI1_MOSI
P27[3]	GPIO_23	KSO_4				QDX_0	PWM_6	SPI1_MISO
P28	GPIO_24	KSO_3					PWM_7	
P29	GPIO_25	KSO_2					I2C0_SCK	UART1_RTS
P30	GPIO_26	KSO_1				UART0_TX	I2C0_SDA	UART1_TX
P31	GPIO_27	KSO_0					I2C1_SCK	UART1_RX
P32	GPIO_28	UART1_RX	KSI_17	KSO_17		QDX_1	I2C1_SDA	UART1_CTS
P33	GPIO_29	UART1_TX				QDX_0		

[3] P27 is not supported by ATM3321/ATM2231.

Table 2.7-2 Multiplexed Pin Function, cont'

Physical Pin	Function						
P0		SHUB1_CLK		SPI1_CLK		XPAON	
P1		SHUB1_CS		SPI1_CS			
P2		SHUB1_MOSI		SPI1_MOSI			
P3	SHUB0_SCK						
P4	SHUB0_SDA						
P5							
P6			SHUB1_CLK				
P7			SHUB1_CS				
P8			SHUB1_MOSI				
P9	SHUB0_CLK	SHUB0_SCK	SHUB1_MISO	SPI0_CLK	PDM_CLK		I2C0_SCK

[3] P27 is not supported by ATM3321/ATM2231.

CONFIDENTIAL

P10	SHUB0_CS	SHUB0_SDA	SHUB0_CLK	SPI0_CS	PDM_IN	XPAON	I2C0_SDA
P11	SHUB0_MOSI	SHUB1_SCK	SHUB0_CS	SPI0_MOSI	UART0_CTS	XPAON	I2C1_SCK
P12		SHUB1_SCK	SHUB0_MOSI		UART0_CTS		
P13	SHUB0_MISO	SHUB1_SDA			UART0_RTS		
P14	QPSI_D3						
P15							
P16							
P17							
P18							
P19							
P20	SHUB0_CLK						
P21	SHUB0_CS						
P22	SHUB0_MOSI						
P23	SHUB0_MISO						
P24	SHUB1_CLK						
P25	SHUB1_CS					XPAON	
P26	SHUB1_MOSI						
P27[3]	SHUB1_MISO						
P28							
P29	SHUB0_SCK						
P30	SHUB0_SDA						
P31	SHUB1_SCK						
P32	SHUB1_SDA						
P33		SHUB1_MISO		SPI1_MISO			

[3] P27 is not supported by ATM3321/ATM2231.

CONFIDENTIAL

The user must program the selection control per pin correctly to connect the pin to the intended functional signal. A set of registers, at_apb_wrpr, encapsulates the selection control bits for all 34 pins. Each pin has a 5-bit register field which is used to select which function is applied to the physical pin. The following table summarizes the mapping between the selection control bits to each pin.

Table 2.7-3 Pin Selection Control to Pin Mapping

Register Name	31:27	26:22	21:17	16:12	11:7	6:2
PIN_SELECTION_A	P0	P1	P2	P3	P4	P5
PIN_SELECTION_B	P6	P7	P8	P9	P10	P11
PIN_SELECTION_C	P12	P13	P14	P15	P16	P17
PIN_SELECTION_D	P18	P19	P20	P21	P22	P23
PIN_SELECTION_E	P24	P25	P26	P27[3]	P28	P29
PIN_SELECTION_F	P30	P31	P32	P33		

[3] P27 is not supported by ATM3321/ATM2231.

How to read the table

Take the first row for instance, the register **PIN_SELECTION_A** encapsulates the selection control for 6 pins (P0 to P5). For example, [16:12] <=> P3. Each IO pin can be configured to be input only, output only, input/output, with or without pull-up with the following signals:

- P_din: input data coming from the pin; this is data coming into the chip
- P_oe: output enable to the pin, which enables/disables the output
- P_ie: input enable to the pin
- P_pu: pullup enable to the pin
- P_dout: output data to the pin; this is data leaving the chip

Once an IO pin is assigned to a function, all its associated control signals are connected to the assigned function or to static logic as appropriate. For example setting **PIN_SELECTION_A**[16:12] to 0 means P3 is assigned to gpio[3]:io (Refer to the Register Specification for pin selection control). Therefore:

- data input of gpio[3] = P_din[3]
- P_oe[3] = output enable control from gpio[3]
- P_ie[3] = input enable control from gpio[3]
- P_pu[3] = pullup enable control from gpio[3]
- P_dout[3] = data output from gpio[3]

For another example setting **PIN_SELECTION_A**[16:12] to 3 (p3_sel=3) means P3 is assigned to uart1_rts:o. Therefore:

- P_din[3] is not assigned to anything because B<uart1_rts> is strictly an output signal, as denoted by the 'o' tag
- P_oe[3] = 1'b1, since the signal is strictly an output
- P_ie[3] = 1'b0
- P_pu[3] = I<p3_pu>, which is a generic pullup control for signal lacking an associated pullup logic; this control signal is defined in the core at_apb_wrpr
- P_dout[3] = driven by uart1_rts

Table 2.7-4 Pin Selection Control and IO Configuration (0-7)

P0	p0_sel=0	p0_sel=1	p0_sel=2	p0_sel=3	p0_sel=4	p0_sel=5	p0_sel=6	p0_sel=7
Function	GPIO_0:io	KSI_19:i	QDZ_1:i	KSO_19:oe	KSO_8:oe	shub1_clk:o	XPAON:o	SPI1_CLK:0
OE control	gpio0.OUTENS	1'b0	1'b0	kso_oe[19]	kso_oe[8]	1'b1	1'b1	1'b1

CONFIDENTIAL

	ET[0]							
IE Control	gpio0.INENABLE_SET[0]	1'b1	1'b1	1'b0	1'b0	1'b0	1'b0	1'b0
PU Control	gpio0.PULLUP_ENABLE_SET[0]	wrpr.pin_pu_A.p0_pu	wrpr.pin_pu_A.p0_pu	wrpr.pin_pu_A.p0_pu	wrpr.pin_pu_A.p0_pu	wrpr.pin_pu_A.p0_pu	wrpr.pin_pu_A.p0_p	wrpr.pin_pu_A.p0_pu
P1	p1_sel=0	p1_sel=1	p1_sel=2	p1_sel=3	p1_sel=4	p1_sel=5		p1_sel=7
Function	GPIO_1:io	KSI_18:i	QDZ_0:i	KSO_18:oe	KSO_9:oe	SHUB1_cs:o		SPI1_CS:0
OE control	gpio0.OUTENSET[1]	1'b0	1'b0	kso_oe[18]	kso_oe[9]	1'b1		1'b1
IE Control	gpio0.INENABLE_SET[1]	1'b1	1'b1	1'b0	1'b0	1'b0		1'b0
PU Control	gpio0.PULLUP_ENABLE_SET[1]	wrpr.pin_pu_A.p1_pu	wrpr.pin_pu_A.p1_pu	wrpr.pin_pu_A.p1_pu	wrpr.pin_pu_A.p1_pu	wrpr.pin_pu_A.p1_pu		wrpr.pin_pu_A.p1_pu
P2	p2_sel=0	p2_sel=1	p2_sel=2	p2_sel=3	p2_sel=4	p2_sel=5	p2_sel=6	
Function	GPIO_2:io	KSI_17:i	QDY_1:i	KSO_17:oe	KSO_10:oe	KSO_8:oe	SHUB1_MOSI:0	
OE control	gpio0.OUTENSET[2]	1'b0	1'b0	kso_oe[17]	kso_oe[10]	kso_oe[8]	1'b1	
IE Control	gpio0.INENABLE_SET[2]	1'b1	1'b1	1'b0	1'b0	1'b0	1'b0	
PU Control	gpio0.PULLUP_ENABLE_SET[2]	wrpr.pin_pu_A.p2_pu	wrpr.pin_pu_A.p2_pu	wrpr.pin_pu_A.p2_pu	wrpr.pin_pu_A.p2_pu	wrpr.pin_pu_A.p2_pu	wrpr.pin_pu_A.p2_p	
P3	p3_sel=0	p3_sel=1	p3_sel=2	p3_sel=3	p3_sel=4		p6_sel=6	p7_sel=7
Function	GPIO_3:io	KSI_16:i	QDY_0:i	UART1_RTS:o	I2C0_SCK:o		KSO_16:oe	KSO_11:oe
OE control	gpio0.OUTENSET[3]	1'b0	1'b0	1'b1	1'b1		kso_oe[16]	kso_oe[11]
IE Control	gpio0.INENABLE_SET[3]	1'b1	1'b1	1'b0	1'b0		1'b0	1'b0
PU Control	gpio0.PULLUP_ENABLE_SET[3]	wrpr.pin_pu_A.p3_pu	wrpr.pin_pu_A.p3_pu	wrpr.pin_pu_A.p3_pu	i2c0.clock_control.clk_pu		wrpr.pin_pu_A.p3_p	wrpr.pin_pu_A.p3_pu
P4	p4_sel=0	p4_sel=1	p4_sel=2	p4_sel=3		p4_sel=5	p4_sel=6	p4_sel=7
Function	GPIO_4:io	KSI_15:io	UART1_RX:i	I2C0_SDA:io		KSO_15:oe	KSO_12:oe	KSO_10:oe
OE control	gpio0.OUTENSET[4]	1'b0	1'b0	i2c0_sda_oe		kso_oe[15]	kso_oe[12]	kso_oe[10]

CONFIDENTIAL

IE Control	**gpio0.INENABLE_SET[4]**	1'b1	1'b1	i2c0_sda_ie		1'b0	1'b0	1'b0
PU Control	gpio0.PULLUP_ENABLE_SET[4]	wrpr.pin_pu_A.p4_pu	wrpr.pin_pu_A.p4_pu	i2c0.outgoing_data.data_pu		wrpr.pin_pu_A.p4_pu	wrpr.pin_pu_A.p4_p	wrpr.pin_pu_A.p4_pu
P5	p5_sel=0	p5_sel=1	p5_sel=2	p5_sel=3				p5_sel=7
Function	GPIO_5:io	KSI_14:i	UART1_TX:o	PWM_0:o				KSO_14:oe
OE control	gpio0.OUTENSET[5]	1'b0	1'b1	1'b1				kso_oe[14]
IE Control	gpio0.INENABLE_SET[5]	1'b1	1'b0	1'b0				1'b0
PU Control	gpio0.PULLUP_ENABLE_SET[5]	wrpr.pin_pu_A.p5_pu	wrpr.pin_pu_A.p5_pu	wrpr.pin_pu_A.p5_pu				wrpr.pin_pu_A.p5_pu
P6	p6_sel=0	p6_sel=1	p6_sel=2	p6_sel=3				
Function	GPIO_6:io	KSI_13:i	UART1_CTS:i	SPI1_CLK:o				
OE control	gpio0.OUTENSET[6]	1'b0	1'b0	1'b1				
IE Control	gpio0.INENABLE_SET[6]	1'b1	1'b1	1'b0				
PU Control	gpio0.PULLUP_ENABLE_SET[6]	wrpr.pin_pu_A.p6_pu	wrpr.pin_pu_A.p6_pu	wrpr.pin_pu_A.p6_pu				
P7	p7_sel=0	p7_sel=1	p7_sel=2	p7_sel=3				
Function	GPIO_7:io	KSI_12:i	PWM_0:o	SPI1_CS:o				
OE control	gpio0.OUTENSET[7]	1'b0	1'b1	1'b1				
IE Control	gpio0.INENABLE_SET[7]	1'b1	1'b0	1'b0				
PU Control	gpio0.PULLUP_ENABLE_SET[7]	wrpr.pin_pu_A.p7_pu	wrpr.pin_pu_A.p7_pu	wrpr.pin_pu_A.p7_pu				
P8	p8_sel=0	p8_sel=1	p8_sel=2	p8_sel=3				
Function	GPIO_8:io	KSI_11:i	PWM_1:o	SPI1_MOSI:o				
OE control	gpio0.OUTENSET[8]	1'b0	1'b1	1'b1				
IE Control	gpio0.INENABLE_SET[8]	1'b1	1'b0	1'b0				

CONFIDENTIAL

PU Control	gpio0.PULLUP_ENABLE_SET[8]	wrpr.pin_pu_B.p8_pu	wrpr.pin_pu_B.p8_pu	wrpr.pin_pu_B.p8_pu				
P9	p9_sel=0	p9_sel=1	p9_sel=2	p9_sel=3				p9_sel=7
Function	GPIO_9:io	KSI_10:i	PWM_2:o	SP1I_MISO:i				KSO_10:oe
OE control	gpio0.OUTENSET[9]	1'b0	1'b1	1'b0				kso_oe[10]
IE Control	gpio0.INENABLE_SET[9]	1'b1	1'b0	1'b1				1'b0
PU Control	gpio0.PULLUP_ENABLE_SET[9]	wrpr.pin_pu_B.p9_pu	wrpr.pin_pu_B.p9_pu	wrpr.pin_pu_B.p9_pu				wrpr.pin_pu_B.p9_pu
P10	p10_sel=0	p10_sel=1	p10_sel=2	p10_sel=3				p10_sel=7
Function	GPIO_10:io	KSI_9:i	SPI0_CLK:o	PWM_1:o				UART0_RX:i
OE control	gpio0.OUTENSET[10]	1'b0	1'b1	1'b1				1'b0
IE Control	gpio0.INENABLE_SET[10]	1'b1	1'b0	1'b0				1'b1
PU Control	gpio0.PULLUP_ENABLE_SET[10]	wrpr.pin_pu_B.p10_pu	wrpr.pin_pu_B.p10_pu	wrpr.pin_pu_B.p10_pu				wrpr.pin_pu_B.p10_pu
P11	p11_sel=0	p11_sel=1	p11_sel=2	p11_sel=3				p11_sel=7
Function	GPIO_11:io	KSI_8:i	SPI0_CS:o	PWM_2:o				UART0_TX:o
OE control	gpio0.OUTENSET[11]	1'b0	1'b1	1'b1				1'b1
IE Control	gpio0.INENABLE_SET[11]	1'b1	1'b0	1'b0				1'b0
PU Control	gpio0.PULLUP_ENABLE_SET[11]	wrpr.pin_pu_B.p11_pu	wrpr.pin_pu_B.p11_pu	wrpr.pin_pu_B.p11_pu				wrpr.pin_pu_B.p11_pu
P12	p12_sel=0	p12_sel=1	p12_sel=2	p12_sel=3	p12_sel=4			
Function	GPIO_12:io	KSI_7:i	QDZ1:i	SPI0_MOSI	I2C1_SCK			
OE control	gpio0.OUTENSET[12]	1'b0	1'b0	1'b1	1'b1			
IE Control	gpio0.INENABLE_SET[12]	1'b1	1'b1	1'b0	1'b0			
PU Control	gpio0.PULLUP_ENABLE_SET[12]	wrpr.pin_pu_B.p12_pu	wrpr.pin_pu_B.p12_pu	wrpr.pin_pu_B.p12_pu	i2c1.clock_control.clk_pu			

CONFIDENTIAL

P13	p13_sel=0	p13_sel=1	p13_sel=2	p13_sel=3	p13_sel=4			
Function	GPIO_13:io	KSI_6:i	QDZ0:i	SPI0_MISO:i	I2C1_SDA:io			
OE control	gpio0.OUTENSET[13]	1'b0	1'b0	1'b0	i2c1_sda_oe			
IE Control	pio0.INENABLE_SET[13]	1'b1	1'b1	1'b1	i2c1_sda_ie			
PU Control	gpio0.PULLUP_ENABLE_SET[13]	wrpr.pin_pu_B.p13_pu	wrpr.pin_pu_B.p13_pu	wrpr.pin_pu_B.p13_pu	i2c1.outgoing_data.data_pu			
P14	p14_sel=0	p14_sel=1						
Function	QSPI_CLK:o	QSPI_D3:io						
OE control	1'b1	qspi_oe[3]						
IE Control	1'b0	qspi_ie[3]						
PU Control	wrpr.pin_pu_B.p14_pu	wrpr.pin_pu_B.p14_pu						
P15	p15_sel=0							
Function	QSPI_CS:o							
OE control	1'b1							
IE Control	1'b0							
PU Control	wrpr.pin_pu_B.p15_pu							
P16	p16_sel=0							
Function	QSPI_D0:io							
OE control	qspi_oe[0]							
IE Control	qspi_ie[0]							
PU Control	wrpr.pin_pu_C.p16_pu							
P17	p17_sel=0	p17_sel=1	p17_sel=2					
Function	QSPI_D1:io	QPSI_CLK:o	QSPI_D0_io					
OE control	qspi_oe[1]	1'b1	qspi_oe[0]					
IE Control	qspi_ie[1]	1'b0	qspi_ie[0]					
PU Control	wrpr.pin_pu_C.p17_pu	wrpr.pin_pu_C.p17_pu	wrpr.pin_pu_C.p17_pu					

CONFIDENTIAL

P18	p18_sel=0	p18_sel=1		p18_sel=3				
Function	QSPI_D2:io	KSI_5:i		GPIO_14:io				
OE control	qspi_oe[2]	1'b0		gpio0.OUTENSET[14]				
IE Control	qspi_ie[2]	1'b1		gpio0.INENABLE_SET[14]				
PU Control	wrpr.pin_pu_C.p18_pu	wrpr.pin_pu_C.p18_pu		gpio0.PULLUP_ENABLE_SET[14]				
P19	p19_sel=0	p19_sel=1		p19_sel=3	p19_sel=4		p19_sel=6	
Function	QSPI_D3:io	KSI_4:i		GPIO_15:io	QPSI_CS:o		QPSI_CLK:o	
OE control	qspi_oe[3]	1'b0		gpio0.OUTENSET[15]	1'b1		1'b1	
IE Control	qspi_ie[3]	1'b1		gpio0.INENABLE_SET[15]	1'b0		1'b0	
PU Control	wrpr.pin_pu_C.p19_pu	wrpr.pin_pu_C.p19_pu		gpio0.PULLUP_ENABLE_SET[15]	wrpr.pin_pu_C.p19_pu		wrpr.pin_pu_C.p19_p	
P20	p20_sel=0	p20_sel=1	p20_sel=2	p20_sel=3				
Function	GPIO_16:io	KSI_3:i	PWM_3:o	SPI0_CLK:o				
OE control	gpio1.OUTENSET[0]	1'b0	1'b1	1'b1				
IE Control	gpio1.INENABLE_SET[0]	1'b1	1'b0	1'b0				
PU Control	gpio1.PULLUP_ENABLE_SET[0]	wrpr.pin_pu_C.p20_pu	wrpr.pin_pu_C.p20_pu	wrpr.pin_pu_C.p20_pu				
P21	p21_sel=0	p21_sel=1	p21_sel=2	p21_sel=3				
Function	GPIO_17:io	KSI_2:i	PWM_4:o	SPI0_CS:o				
OE control	gpio1.OUTENSET[1]	1'b0	1'b1	1'b1				
IE Control	gpio1.INENABLE_SET[1]	1'b1	1'b0	1'b0				
PU Control	gpio1.PULLUP_ENABLE_SET[1]	wrpr.pin_pu_C.p21_pu	wrpr.pin_pu_C.p21_pu	wrpr.pin_pu_C.p21_pu				
P22	p22_sel=0	p22_sel=1	p22_sel=2	p22_sel=3				
Function	GPIO_18:io	KSI_1:i	PWM_5:o	SPI0_MOSI:o				

CONFIDENTIAL

OE control	gpio1.OUTENSET[2]	1'b0	1'b1	1'b1				
IE Control	gpio1.INENABLE_SET[2]	1'b1	1'b0	1'b0				
PU Control	gpio1.PULLUP_ENABLE_SET[2]	wrpr.pin_pu_C.p22_pu	wrpr.pin_pu_C.p22_pu	wrpr.pin_pu_C.p22_p				
P23	p23_sel=0	p23_sel=1	p23_sel=2	p23_sel=3				
Function	GPIO_19:io	KSI_0:i	PWM_6:o	SPI0_MISO:i				
OE control	gpio1.OUTENSET[3]	1'b0	1'b1	1'b0				
IE Control	gpio1.INENABLE_SET[3]	1'b1	1'b0	1'b1				
PU Control	gpio1.PULLUP_ENABLE_SET[3]	wrpr.pin_pu_C.p23_pu	wrpr.pin_pu_C.p23_pu	wrpr.pin_pu_C.p23_pu				
P24	p24_sel=0	p24_sel=1	p24_sel=2	p24_sel=3	p24_sel=4			
Function	GPIO_20:io	KSO_7:oe	QDY_1:i	SPI1_CLK:o	PWM_3:o			
OE control	gpio1.OUTENSET[4]	kso_oe[7]	1'b0	1'b1	1'b1			
IE Control	gpio1.INENABLE_SET[4]	1'b0	1'b1	1'b0	1'b0			
PU Control	gpio1.PULLUP_ENABLE_SET[4]	wrpr.pin_pu_D.p24_pu	wrpr.pin_pu_D.p24_pu	wrpr.pin_pu_D.p24_pu	wrpr.pin_pu_D.p24_pu			
P25	p25_sel=0	p25_sel=1	p25_sel=2	p25_sel=3	p25_sel=4			
Function	GPIO_21:io	KSO_6:oe	QDY_0:i	SPI1_CS:o	PWM_4:o			
OE control	gpio1.OUTENSET[5]	kso_oe[6]	1'b0	1'b1	1'b1			
IE Control	gpio1.INENABLE_SET[5]	1'b0	1'b1	1'b0	1'b0			
PU Control	gpio1.PULLUP_ENABLE_SET[5]	wrpr.pin_pu_D.p25_pu	wrpr.pin_pu_D.p25_pu	wrpr.pin_pu_D.p25_pu	wrpr.pin_pu_D.p25_pu			
P26	p26_sel=0	p26_sel=1	p26_sel=2	p26_sel=3	p26_sel=4			
Function	GPIO_22:io	KSO_5:oe	DQX_1:i	SPI1_MOSI:o	PWM_5:o			
OE control	gpio1.OUTENSET[6]	kso_oe[5]	1'b0	1'b1	1'b1			
IE Control	gpio1.INENABL	1'b0	1'b1	1'b0	1'b0			

CONFIDENTIAL

	E_SET[6]							
PU Control	gpio1.PULLUP_ENABLE_SET[6]	wrpr.pin_pu_D.p26_pu	wrpr.pin_pu_D.p26_pu	wrpr.pin_pu_D.p26_pu	wrpr.pin_pu_D.p26_p			
P27[3]	p27_sel=0	p27_sel=1	p27_sel=2	p27_sel=3	p27_sel=4			
Function	GPIO_23:io	KSO_4:oe	QDX_0:i	SPI1_MISO:i	PWM_6:o			
OE control	gpio1.OUTENSET[7]	kso_oe[4]	1'b0	1'b0	1'b1			
IE Control	gpio1.INENABLE_SET[7]	1'b0	1'b1	1'b1	1'b0			
PU Control	gpio1.PULLUP_ENABLE_SET[7]	wrpr.pin_pu_D.p27_pu	wrpr.pin_pu_D.p27_pu	wrpr.pin_pu_D.p27_pu	wrpr.pin_pu_D.p27_pu			
P28	p28_sel=0	p28_sel=1	p28_sel=2					
Function	GPIO_24:io	KSO_3:oe	PWM_7:o					
OE control	gpio1.OUTENSET[8]	kso_oe[3]	1'b1					
IE Control	gpio1.INENABLE_SET[8]	1'b0	1'b0					
PU Control	gpio1.PULLUP_ENABLE_SET[8]	wrpr.pin_pu_D.p28_pu	wrpr.pin_pu_D.p28_pu					
P29	p29_sel=0	p29_sel=1	p29_sel=2	p29_sel=3				
Function	GPIO_25:io	KSO_2:oe	I2C0_SCK:o	UART1_RTS:o				
OE control	gpio1.OUTENSET[9]	kso_oe[2]	1'b1	1'b1				
IE Control	gpio1.INENABLE_SET[9]	1'b0	1'b0	1'b0				
PU Control	gpio1.PULLUP_ENABLE_SET[9]	wrpr.pin_pu_D.p29_pu	i2c0.clock_control.clk_pu	wrpr.pin_pu_D.p29_pu				
P30	p30_sel=0	p30_sel=1	p30_sel=2	p30_sel=3			p30_sel=6	
Function	GPIO_26:io	KSO_1:oe	I2C0_SDA:io	UART1_TX:o			UART0_TX:o	
OE control	gpio1.OUTENSET[10]	kso_oe[1]	i2c0_sda_oe	1'b1			1'b1	
IE Control	gpio1.INENABLE_SET[10]	1'b0	i2c0_sda_ie	1'b0			1'b0	
PU Control	gpio1.PULLUP_ENABLE_SET[1	wrpr.pin_pu_D.p30_pu	i2c0.outgoing_data.data_	wrpr.pin_pu_D.p30_pu			wrpr.pin_pu_D.p30_pu	

CONFIDENTIAL

	0]		pu					
P31	p31_sel=0	p31_sel=1	p31_sel=2	p31_sel=3			p31_sel=6	
Function	GPIO_27:io	KSO_0:oe	I2C1_SCK:o	UART1_RX:i			SHUB1_SCK:o	
OE control	gpio1.OUTENSET[11]	kso_oe[0]	1'b1	1'b0			1'b1	
IE Control	gpio1.INENABLE_SET[11]	1'b0	1'b0	1'b1			1'b0	
PU Control	gpio1.PULLUP_ENABLE_SET[11]	wrpr.pin_pu_D.p31_pu	i2c1.clock_control.clk_pu	wrpr.pin_pu_D.p31_pu			wrpr.pin_pu_D.p31_pu	
P32	p32_sel=0	p32_sel=1	p32_sel=2	p32_sel=3	p32_sel=4	p32_sel=5		p32_sel=7
Function	GPIO_28:io	UART1_RX:i	QDX_1:i	KSI_17:i	I2C1_SDA:io	UART1_CTS:i		KSO_17:oe
OE control	gpio1.OUTENSET[12]	1'b0	1'b0	1'b0	i2c1_sda_oe	1'b0		kso_oe[17]
IE Control	gpio1.INENABLE_SET[12]	1'b1	1'b1	1'b1	i2c1_sda_ie	1'b1		1'b0
PU Control	gpio1.PULLUP_ENABLE_SET[12]	wrpr.pin_pu_E.p32_pu	wrpr.pin_pu_E.p32_pu	wrpr.pin_pu_E.p32_pu	i2c1.outgoing_data.data_pu	wrpr.pin_pu_E.p32_pu		wrpr.pin_pu_E.p32_pu
P33	p33_sel=0	p33_sel=1	p33_sel=2					p33_sel=7
Function	GPIO_29:io	UART1_TX:o	QDX_0:o					SHUB1_MISO:i
OE control	gpio1.OUTENSET[13]	1'b1	1'b0					1'b0
IE Control	gpio1.INENABLE_SET[13]	1'b0	1'b1					1'b1
PU Control	gpio1.PULLUP_ENABLE_SET[13]	wrpr.pin_pu_E.p33_pu	wrpr.pin_pu_E.p33_pu					wrpr.pin_pu_E.p33_pu

[3] P27 is not supported by ATM3321/ATM2231.

Table 2.7-4 Pin Selection Control and IO Configuration (8-15)

P2	p2_sel=8							
Function	SPI1_MOSI:o							
OE control	1'b1							
IE Control	1'b0							

CONFIDENTIAL

PU Control	wrpr.pin_pu_A.p2_pu							
P3	p3_sel=8	p3_sel=9						
Function	KSO_9:oe	SHUB0_SCK:o						
OE control	kso_oe[9]	1'b1						
IE Control	1'b0	1'b0						
PU Control	wrpr.pin_pu_A.p3_pu	wrpr.pin_pu_A.p3_pu						
P4	p4_sel=8		p4_sel=10					
Function	SHUB0_SDA:io		QSPI_D0:io					
OE control	shub0_sda_oe		qspi_oe[0]					
IE Control	shub0_sda_ie		qspi_ie[0]					
PU Control	shub0_sda_pu		wrpr.pin_pu_A.p4_pu					
P5	p5_sel=8	p5_sel=9		p5_sel=11				
Function	KSO_13:oe	KSO_11:oe		QSPI_CS:O				
OE control	kso_oe[13]	kso_oe[11]		1'b1				
IE Control	1'b0	1'b0		1'b0				
PU Control	wrpr.pin_pu_A.p5_pu	wrpr.pin_pu_A.p5_pu		wrpr.pin_pu_A.p5_pu				
P6	p6_sel=8	p6_sel=9	p6_sel=10	p6_sel=11		p6_sel=13		
Function	KSO_13:oe	KSO_14:oe	KSO_12:oe	SHUB1_CLK:o		QSPI_D1:io		
OE control	kso_oe[13]	kso_oe[14]	kso_oe[12]	1'b1		qspi_oe[1]		
IE Control	1'b0	1'b0	1'b0	1'b0		qspi_ie[1]		
PU Control	wrpr.pin_pu_A.p6_pu	wrpr.pin_pu_A.p6_pu	wrpr.pin_pu_A.p6_pu	wrpr.pin_pu_A.p6_pu		wrpr.pin_pu_A.p6_pu		
P7	p7_sel=8	p7_sel=9	p7_sel=10	p7_sel=11		p7_sel=13		
Function	KSO_12:ie	KSO_15:oe	KSO_13:oe	SHUB1_CS:o		QSPI_D2:io		
OE control	kso_oe[12]	kso_oe[15]	kso_oe[13]	1'b1		qspi_oe[2]		
IE Control	1'b0	1'b0	1'b0	1'b0		qspi_ie[2]		

CONFIDENTIAL

PU Control	wrpr.pin_pu_A.p7_pu	wrpr.pin_pu_A.p7_pu	wrpr.pin_pu_A.p7_pu	wrpr.pin_pu_A.p7_pu		wrpr.pin_pu_A.p7_pu		
P8	p8_sel=8		p8_sel=10	p8_sel=11	p8_sel=12			
Function	KSO_11”oe		KSO_16:oe	KSO_14:oe	SHUB1_MOSI:o			
OE control	kso_oe[11]		kso_oe[16]	kso_oe[14]	1'b1			
IE Control	1'b0		1'b0	1'b0	1'b0			
PU Control	wrpr.pin_pu_B.p8_pu		wrpr.pin_pu_B.p8_pu	wrpr.pin_pu_B.p8_pu	wrpr.pin_pu_B.p8_pu			
P9	p9_sel=8	p9_sel=9				p9_sel=13	p9_sel=14	p9_sel=15
Function	SPI0_CLK:o	I2C0_SCK:o				KSO_17:oe	KSO_15:oe	SHUB0_clk:o
OE control	1'b1	1'b1				kso_oe[17]	kso_oe[15]	1'b1
IE Control	1'b0	1'b0				1'b0	1'b0	1'b0
PU Control	wrpr.pin_pu_B.p9_pu	i2c0.clock_control.clk_pu				wrpr.pin_pu_B.p9_pu	wrpr.pin_pu_B.p9_pu	wrpr.pin_pu_B.p9_pu
P10	p10_sel=8	p10_sel=9	p10_sel=10				p10_sel=14	p10_sel=15
Function	KSO_9:oe	SPI0_CS:o	I2C0_SDA:o				KSO_18:oe	kso_16_oe
OE control	kso_oe[9]	1'b1	i2c0_sda_oe				kso_oe[18]	kso_oe[16]
IE Control	1'b0	1'b0	i2c0_sda_ie				1'b0	1'b0
PU Control	wrpr.pin_pu_B.p10_pu	wrpr.pin_pu_B.p10_pu	i2c0.outgoing_data.data_pu				wrpr.pin_pu_B.p10_pu	wrpr.pin_pu_B.p10_pu
P11	p11_sel=8	p11_sel=9	p11_sel=10			p11_sel=13	p11_sel=14	p11_sel=15
Function	KSO_8:oe	SPI0_MOSI:O	I2C1_SCK:o			KSO_19:oe	KSO_17:oe	UART0_CTS:i
OE control	kso_oe[8]	1'b1	1'b1			kso_oe[19]	kso_oe[17]	1'b0
IE Control	1'b0	1'b0	1'b0			1'b0	1'b0	1'b1
PU Control	wrpr.pin_pu_B.p11_pu	wrpr.pin_pu_B.p11_pu	i2c1.clock_control.clk_pu			wrpr.pin_pu_B.p11_pu	wrpr.pin_pu_B.p11_pu	wrpr.pin_pu_B.p11_pu
P12		p12_sel=9	p12_sel=10	p12_sel=11	p12_sel=12			
Function		KSO_18:oe	UART0_CTS:i	SHUB0_MOSI:o	SHUB1_SCK:o			
OE control		kso_oe[18]	1'b0	1'b1	1'b1			

CONFIDENTIAL

IE Control		1'b0	1'b1	1'b0	1'b0			
PU Control		wrpr.pin_pu_B.p12_pu	wrpr.pin_pu_B.p12_pu	wrpr.pin_pu_B.p12_pu	wrpr.pin_pu_B.p12_pu			
P13		p13_sel=9	p13_sel=10	p13_sel=11	p13_sel=12			
Function		KSO_19:oe	UART0_RTS:o	SHUB0_MISO:i	SHUB1_SDA:io			
OE control		kso_oe[19]	1'b1	1'b0	shub1_sda_oe			
IE Control		1'b0	1'b0	'b1	hub1_sda_ie			
PU Control		wrpr.pin_pu_B.p13_pu	wrpr.pin_pu_B.p13_pu	wrpr.pin_pu_B.p13_pu	shub1_sda_pu			
P20		p20_sel=9			p20_sel=12			
Function		qspi_do:io			shub0_clk:O			
OE control		qspi_oe[0]			1'b1			
IE Control		qspi_ie[0]			1'b0			
PU Control		wrpr.pin_pu_C.p20_pu			wrpr.pin_pu_C.p20_pu			
P21			p21_sel=10	p21_sel=11				
Function			UART0_CTS:i	SHUB0_CS:o				
OE control			1'b0	1'b1				
IE Control			1'b1	1'b0				
PU Control			wrpr.pin_pu_C.p21_pu	wrpr.pin_pu_C.p21_pu				
P22		p22_sel=9	p22_sel=10			p22_sel=13		
Function		UART0_RX:i	QPSI_D1:io			SHUB0_MOSI:o		
OE control		1'b0	qspi_oe[1]			1'b1		
IE Control		1'b1	qspi_ie[1]			1'b0		
PU Control		wrpr.pin_pu_C.p22_pu	wrpr.pin_pu_C.p22_pu			wrpr.pin_pu_C.p22_pu		
P23		p23_sel=9	p23_sel=10			p23_sel=13		

CONFIDENTIAL

Function		UART0_TX:o	QSPI_D2:io			SHUB0_MISO:i		
OE control		1'b1	qspi_oe[2]			1'b0		
IE Control		1'b0	qspi_ie[2]			1'b1		
PU Control		wrpr.pin_pu_C.p23_pu	wrpr.pin_pu_C.p23_pu			wrpr.pin_pu_C.p23_pu		
P24			p24_sel=10			p24_sel=13	p24_sel=14	
Function			QPSI_D3:io			UART0_RTS:o	SHUB1_CLK:o	
OE control			qspi_oe[3]			1'b1	1'b1	
IE Control			qspi_ie[3]			1'b0	1'b0	
PU Control			wrpr.pin_pu_D.p24_pu			wrpr.pin_pu_D.p24_pu	wrpr.pin_pu_D.p24_pu	
P25			p25_sel=10		p25_sel=12		p25_sel=14	
Function			UART0_RX:i		SHUB1_CS:o		PDM_IN:i	
OE control			1'b0		1'b1		1'b0	
IE Control			1'b1		1'b0		1'b1	
PU Control			wrpr.pin_pu_D.p25_pu		wrpr.pin_pu_D.p25_pu		wrpr.pin_pu_D.p25_pu	
P26				p26_sel=11				
Function				SHUB1_MOSI:o				
OE control				1'b1				
IE Control				1'b0				
PU Control				wrpr.pin_pu_D.p26_pu				
P27[3]				p27_sel=11				
Function				SHUB1_MISO:i				
OE control				1'b0				
IE Control				1'b1				
PU Control				wrpr.pin_pu_D.p27_pu				

CONFIDENTIAL

P30	p30_sel=8							
Function	SHUB0_SDA:io							
OE control	shub0_sda_oe							
IE Control	shub0_sda_ie							
PU Control	shub0_sda_pu							
P32	p32_sel=8							
Function	SHUB1_SDA:io							
OE control	shub1_sda_oe							
IE Control	shub1_sda_ie							
PU Control	shub1_sda_pu							
P33		p33_sel=9						
Function		SPI1_MISO:i						
OE control		1'b0						
IE Control		1'b1						
PU Control		wrpr.pin_pu_E.p33_pu						

[3] P27 is not supported by ATM3321/ATM2231.

Table 2.7-4 Pin Selection Control and IO Configuration (16-23)

P9	p9_sel=16	p9_sel=17		p9_sel=19				
Function	SHUB0_SCK:o	SHUB1_MISO:i		PDM_CLK:o				
OE control	1'b1	1'b0		1'b1				
IE Control	1'b0	1'b1		1'b0				
PU Control	wrpr.pin_pu_B.p9_pu	wrpr.pin_pu_B.p9_pu		wrpr.pin_pu_B.p9_pu				
P10	p10_sel=16	p10_sel=17	p10_sel=18		p10_sel=20	p10_sel=21		

CONFIDENTIAL

Function	SHUB0_CS:o	SHUB0_SDA:Io	SHUB0_CLK:o		PDM_IN:i	XPAON:o		
OE control	1'b1	shub0_sda_oe	1'b1		1'b0	1'b1		
IE Control	1'b0	shub0_sda_ie	1'b0		1'b0	1'b0		
PU Control	wrpr.pin_pu_B.p10_pu	shub0_sda_pu	wrpr.pin_pu_B.p10_pu		wrpr.pin_pu_B.p10_pu	wrpr.pin_pu_B.p10_pu		
P11	p11_sel=16	p11_sel=17	p11_sel=18	p11_sel=19				
Function	SHUB0_MOSI:o	SHUB1_SCK:o	SHUB0:CS:o	XPAON:o				
OE control	1'b1	1'b1	1'b1	1'b1				
IE Control	1'b0	1'b0	1'b0	1'b0				
PU Control	wrpr.pin_pu_B.p11_pu	wrpr.pin_pu_B.p11_pu	wrpr.pin_pu_B.p11_pu	wrpr.pin_pu_B.p11_pu				
P24	p24_sel=16							
Function	PDM_CLK:o							
OE control	1'b1							
IE Control	1'b0							
PU Control	wrpr.pin_pu_D.p24_pu							
P25	p25_sel=16							
Function	XPAON_o							
OE control	1'b1							
IE Control	1'b0							
PU Control	wrpr.pin_pu_D.p25_pu							

CONFIDENTIAL

2.8 PSEQ

Program Sequencer (PSEQ) provides mechanism for the user application to enter and to leave the low power states: Retention (Retain All), Hibernation, and SoC Off.

Wake from Hibernation or Retention is achieved from a variety of possible sources. Wake can be from a timer expiring, activity detected on GPIO's, activity detected on the keyboard, activity detected on the mouse, the sensor hub reading

measurements crossing a threshold, the wake-up receiver receiving the right pattern over RF, and the detection of a connection over the SWD interface. Any combination of these seven sources is possible. Use **hib_wake_mask** or **retain_all_wake_mask** register to set the valid wake-up sources.

All selections about how to wake need to be programmed before the transition into the low power state is triggered. A transition into hibernation is triggered by setting PSEQ **ctrl0**.*req_hibernate_state* and a transition into Retention is triggered by setting **ctrl0**.*req_retain_all_state*. Prior to triggering the transition into the low power state, the user application should first disable all interrupt sources except the PSEQ at_apb_pseq's interrupt. Immediately after triggering the transition into the low power state, the user application should place the M0 in wait for interrupt (WFI) mode.

- Wake from timer is done by setting either **hib_wake_mask**.*watch_cntdown* or **retain_all_wake_mask**.*watch_cntdown* and programming init_count_down register to a non-zero value. This counter operates at 32 kHz and will trigger the appropriate power up sequence when it reaches zero.

- Wake from GPIO is done by setting either **hib_wake_mask**.*watch_gpio* or **retain_all_wake_mask**.*watch_gpio* and programming **hib_wake_mask**, **gpio_wake_type**, gpio_wake_pol, and/or **gpio_wake_both_edges** registers.

- Wake from activity on keyboard and/or activity on mouse is done by setting **hib_wake_mask**.*watch_ksm*, **retain_all_wake_mask**.*watch_ksm*, **hib_wake_mask**.*watch_qjdec*, **retain_all_wake_mask**.*watch_qdec*. The corresponding cores (at_apb_ksm for keyboard and at_apb_qdec for mouse) will have to have been configured for correct operation prior to triggering the transition into the low power state. The activity detection does require some additional top level logic to remain powered. If the application does not use these devices or does not use them to generate a wake event, these cores can be powered down indefinitely using **ksmqdec_control** register.

- Wake from sensor hub is done by setting either **hib_wake_mask**.*watch_shub* or **retain_all_wake_mask**.*watch_shub*. The corresponding core (at_apb_pseq) will have to have been configured for correct operation prior to triggering the transition into the low power state. The sensor hub does require some additional top level logic to remain powered. If the application does not use the sensor hub, at_apb_shub can be powered down indefinitely using **sensor_hub_control** register.

- Wake from wake-up receiver is done by setting **hib_wake_mask**.*watch_wurx1*, **hib_wake_mask**.*watch_wurx0*, **retain_all_wake_mask**.*watch_wurx1*, or **retain_all_wake_mask**.*watch_wurx0*. The wake-up receiver will have to have been configured for correct operation prior to triggering the transition into the low power state. The wake-up receiver does require some circuitry to remain powered. If the application does not use the wake-up receiver, it can be powered down indefinitely using **wurx_config** register.

- Wake from the detection of a connection over the SWD interface is enabled by default. Refer to ARM Debug Interface Specification v5 for more information on the SWD. If either CSYSPWRUPREQ or CDBGPWRUPREQ is set in the SWD interface, a wake will occur. If this behavior is not desired, CSYSPWRUPREQ and CDBGPWRUPREQ can be cleared or PSEQ **ctrl0**.*block_dbg_wake* bit can be set.

The user application, when returning from a lower power state, can determine why the wake occured by querying status. The register **current_real_time** can be used to track how much real time has elapsed. The real time counter operates from CLK_LPC (32 kHz) and will run continuously and is independent of the power state. The register **persistent0** is a general scratch pad register available for software use and may be of use during debug. Its contents persist through all power states and the contents have no impact on the hardware.

This core also generates PMU signals to indicate the amount of energy available from the battery or harvester. Energy4TX signal is used by Atmosic software for brownout support. After the SOC is programmed to enter any low power state, the software firstly checks the status of energy4TX. If it is asserted, the SOC will enter its preprogrammed low power state. If it is deasserted, indicating not enough energy from the battery or harvester, the SOC will be programmed to enter SoC Off

CONFIDENTIAL

state, gated by enough energy indication. For SOC with harvesting capabilities, it will be brought out of SoC Off when the harvester is restored to enough energy. For SOC without harvesting capabilities, it will stay in SoC Off.

2.9 Peripherals and IO

The following peripherals are supported by the SoC.

- **GPIO**
  There are up to 29 GPIOs available on the ATM2231/ATM3231 56-pin 7x7mm package, up to 30 GPIOs available on the ATM2201/ATM2221/ATM3201/ATM3221 64-pin 6x6mm package, and up to 16 GPIOs in the 40-pin 5x5mm package. There are up to 12 GPIOs in the ATM3202/ATM2202. In addition to providing general purpose read and write access, the GPIO can also be used to generate interrupts and to wake the ATM2/ATM3 from low power states.

  Control registers (e.g. output enable, input enable, interrupt type) are set and cleared by independent registers. The readback on these set/clear registers reflect the state of the control registers. For instance, to change the state of a GPIO's output enable, the user can program the corresponding bit in the **OUTENSET** or **OUTENCLR** registers to 1 at any time and the latest programming of the two registers determines whether that GPIO is an output. A readback of either the **OUTENSET** or **OUTENCLR** register will return the same value. The GPIO can toggle at a frequency of up to 4 MHz (clk_mpc/4) as it takes two cycles to write 1 and two cycles to write 0 on an AHB register. The AHB GPIO provides programmable interrupt generation features. Three registers control this functionality and each register has separate set and clear addresses. Each bit of the IO pins can be configured to generate interrupts based on these three registers:

Table 2.9-1 Interrupt Registers

Interrupt_enable	Interrupt_polarity	Interrupt_type	Interrupt feature
0	-	-	Disabled
1	0	0	Low-level
1	0	1	Falling edge
1	1	0	High-level
1	1	1	Rising edge

  Each register is controlled by a pair of software-controlled registers. One is used to set; the other one is used to clear. After an interrupt is triggered, the corresponding bit in the GPIO **INTSTATUS_CLEAR** register is set. This also causes the corresponding bit of internal GPIOINT[15:0] signal to be asserted. As a result, the combined interrupt internal signal, COMBINT, is also asserted. The interrupt status can be cleared by using an interrupt handler that writes 1 to the corresponding bit of **INTSTATUS_CLEAR** register..

  The mapping from GPIO's to physical pins is provided by the PINMUX software tool. Each physical pin has multiple functional signals. The 30 GPIO pins are mapped to 30 out of the total 34 physical pins (P0,... P33), skipping P14 - P17. The module, at_ahb_gpio, is instantiated twice. One core, assigned to address space 0x40010000 - 0x40010FFC, encapsulates the control of GPIO[0] to GPIO[15]. The other core, assigned to address space 0x40011000 - 0x40011FFC, encapsulates the control of GPIO[16] to GPIO[29].

2.9.1 Controlling input enable and pull up enable

The physical pins have both input enable and pull up enable signals. Disabling the input receivers can save power when the state of the GPIO is unknown. These two registers can be used to change the state of the pullup enable.

CONFIDENTIAL

PULLUP_ENABLE_SET : to enable pullup per pin

PULLUP_ENABLE_CLR : to disable pullup per pin

These two registers can be used to change the state of the input enable.
INENABLE_SET : to enable the input driver per pin

INENABLE_CLR : to disable the input driver per pin (reducing power for GPIO whose input is not used)

Example: Configure P13 to be GPIO[13] and output only

- Select GPIO[13] on P13 by programming WRPR **PIN_SELECTION_C**[26:22] to 0.
- Program GPIO0 **OUTENSET**[13] to 1 and **DATAOUT**[13] to 0/1 as desired.

Example: Configure P23 to be GPIO[19] and input only

- Select GPIO[19] on P23 by programming **PIN_SELECTION_D**[6:2] to 0.
- Program GPIO0 **OUTENCLR**[3] to 1 and **INENABLE_SET**[3] to 1. The user samples the input data from DATA[3].

- **I2C**

  There are two identical I2C masters. Software preloads the transaction and then initiates the hardware controller. Software can either poll for completion or respond to the completion interrupt. The I2C core supports clock frequencies between 3.9 kHz-4 MHz by setting *clkdiv* in the clock_control register (frequency = 16MHz/4*(*clkdiv*+1)). To Read/Write slave devices use following steps:

  - Enable clock, toggle reset to I2C core(s) by writing to **APB0_CTRL** register for i2c0, **APB1_CTRL** register for i2c1 under the WRPR1 section of the register specification.
  - Read I2C0 **core_id** register and it should match 32'h4932 4320 (defined in register spec). Setup I2C pins by writing to PIN_SELECTION registers in at_apb_wrpr. If set up to masters, write to appropriate I2C transaction_setup and clock_control registers (clkdiv, ack, tail, head ptr etc). If writing multiple bytes (burst write), make sure head, tail bytes are set correctly.
  - For Writes, place the data in I2C **outgoing_data** register. Set go bit in **transaction_setup** register, this initiates I2C FSM and drives txn.
  - There are two ways to signal transaction end
    - Polling - Read status using transaction_status register
    - Interrupt - I2C core generates interrupt which can be used by M0. Interrupt can be masked, set, cleared with appropriate registers
  - For reads, readback data from I2C **incoming_data** registers.

- **Serial Peripheral Interface (SPI)**

  SPI interface allows software to preload the transaction, initiate it, and then either poll for completion or respond to the completion interrupt. Opcode, transaction type, data, and number of bytes are all software programmable. The hardware will serialize and sample incoming data as required by the protocol. The SPI core is an APB slave.

  The Soc has two SPI masters, providing control signals (Clk, CS, MOSI) to slave, and receiving data through signal MISO. The SPI core is fixed to mode 0 (CPOL=0, CPHA=0) and supports clock frequencies between 7.8 kHz-8 MHz by setting *clkdiv* in the transaction_setup register (frequency = 16MHz/2*(*clkdiv*+1)). To Read/Write slave devices use following steps:

  1. Enable clock, toggle reset to SPI core(s) by writing to **APB7_CTRL**(spi0), **APB13_CTRL**(spi1) register in WRPR.
  2. Read SPI **core_id** and it should match 32'h5350 4920 as defined in Register Specification.
  3. Setup SPI pins by writing to Pin Selection registers in at_apb_wrpr. If you plan to setup two slaves, write to appropriate pin selection registers.
  4. Write to transaction setup register: *opcode*, *clkdiv*, *num_data_bytes*, *dummy_cycles*.
  5. Set **Start** bit in **transaction_setup** register, this initiates SPI FSM and drives outputs.
  6. For Writes, place the data to write in register: **data_bytes_lower**, **data_bytes_upper**.

CONFIDENTIAL

7. There are two ways to signal transaction end
    - Polling - Read status using transaction_status register
    - Interrupt - Core generates int which can be used by M0 core. Interrupt can be masked, set, cleared with appropriate registers
8. For reads, readback data from lower, upper registers.

- **Quad serial peripheral interface (QSPI)**

  There is one quad SPI master port and it is intended to be connected to a flash device at up to 32 MHz (when needed). The core is an APB slave. It generates control/data signals (clk, cs, d[3:0]), which are used to connect to external flash.

  This QSPI supports the following features:

  1. Programmable commands and modes-(Single/Dual/Quad instruction/data, XIP) to support multiple flash vendors.
  2. 4/8/16/32 MHz speeds supported - Programmed through QSPI **remote_ahb_setup** register. 32 MHz mode is referred to as hyper mode
  3. Cache (256x48 bit) accessible through AHB Single/QSPI interface. This speeds up flash reads by 10 times.
  4. Multiple ways to access flash memory - Single/Dual/Quad APB, Single/Dual/Quad AHB. This helps with Flash bring-up, debug and faster access. To Read/Write flash devices with QSPI, use the following steps: (Also refer to vendor provided datasheet for opcodes, ID values, register fields, definitions etc).

  Single SPI APB:

  1. Enable clock, toggle reset to SPI2/QSPI core(s) by writing to **APB11_CTRL**(SPI2), **APB15_CTRL**(QSPI) register in WRPR.
  2. Enable clock, toggle reset to PSEQ by writing to **APB8_CTRL**(PSEQ). Turn on clkhpc bit (32 MHz) by writing to **XTAL_BITS1** register.
  3. Setup QSPI pins by writing to **PIN_SELECTION** registers in at_apb_wrpr.
  4. Read spi2/qspi **core_id** and it should match 32'h5350 4920, 32'h5153 5049 respectively as defined in Register Specification.
  5. Read flash id (RDID) by setting appropriate QSPI registers in SPI2. Make sure it matches vendor provided values. Vendors also provide memory type, density when **num_data_bytes** is set to 3.
  6. To write to external flash:
     - Set Enable (WREN) opcode (refer to flash vendors' documentation) in **transaction_setup** register in SPI2 (*opcode*, *clkdiv*, *num_data_bytes*, *dummy_cycles*).
     - There are 2 ways to signal transaction end
       i. Polling - Issue Read status (RDSR) opcode to read from flash.
       ii. Interrupt - Core generates int which can be used by M0 core. Interrupt can be masked, set, cleared with appropriate registers
     - Set Page Program (PP) opcode in **transaction_setup** register in SPI2 (*opcode*, *clkdiv*, *num_data_bytes*, *dummy_cycles*). Wait for transaction to complete
  7. To read from flash:
     - Set Enable (WREN) opcode in **transaction_setup** register in SPI2 (*opcode*, *clkdiv*, *num_data_bytes*, *dummy_cycles*).
     - Wait for transaction to end
     - Set Read (READ) opcode in **transaction_setup** register in SPI2 (*opcode*, *clkdiv*, *num_data_bytes*, *dummy_cycles*). Wait for transaction to complete

  QSPI Single/Quad AHB:

  Using QSPI AHB with Cache, Hyper(32 MHz) and high performance provides the best flash performance.

CONFIDENTIAL

1. Do steps 1-5 from Single SPI APB.
2. Setup Quad mode in external flash memory:
3. Setup QSPI **remote_ahb setup** registers (1-6), and **transaction_setup** register. Special bits need to be set for following:
    i. *Enable Cache* bit for faster Flash reads
    ii. Enable *slip_half_cycle* for high performance (for devices that allow the master to send address in Quad mode).
4. Write/Read flash memory.

To bring up flash and get the best performance, follow these steps:

1. Read the flash datasheet and gather opcodes, register definitions, addresses etc.
2. Setup **pin_selection_**x for connection to external flash. Multiple configurations are available based on package.
3. Setup Single SPI (spi2) access by writing to *remote_spi_has_control* bit. Read ID, write/read memory. Run at different speeds and make sure the flash is working. This is for initial bring-up.
4. Setup Quad mode by writing QE bit in the status register of the flash. Read ID in Quad mode, write/read memory. Setup Dual mode and repeat the step. This is to ensure Quad, Dual SPI works correctly. Try setting *Hyper*, *enable_performance_mode* bits and repeat.
5. Setup AHB by writing to *remote_ahb_qspi_has_control* bit:

    a. Read ID, write/read memory.

    b. Enable Cache and repeat step a. You will notice improved read time after a few flash reads.

    c. Set *Hyper*, *enable_performance_mode* bits and repeat.

- **UART (Universal Asynchronous Receiver/Transmitter)**

  There are two UART cores with hardware flow control present on the ATM2/ATM3. UART0 is configured (by Atmosic software) to be used for HCI type applications with flow control and UART1 primarily for debug messaging, therefore without flow control.

  The UART is an APB slave. The functionality, except for the Atmosic's modifications, is covered in ARM's Cortex-M System Design Kit Rev r1p0. The UART core supports standard baud rates up to 460800 by setting the **bauddiv** register. It only supports one stop bit and no parity bits. The modifications implement Tx/Rx buffers and the hardware flow feature (nUARTCTS and nUARTRTS). In this implementation, the Rx and Tx are each equipped with a 16-deep FIFO buffer. The Rx FIFO acts as a buffer for the incoming data bytes. The CPU is interrupted when the RX FIFO has waited long enough to have received **rx_lwm** items , starting from the first item received. In addition , when the Rx FIFO is occupied with 13 bytes and beyond, the output nUARTRTS is asserted low to warn the transmitting uart. The Tx FIFO acts as an outgoing buffer so that the CPU can fill up multiple bytes of data at a time and go to attend other tasks while the slow transmission is draining the buffer. The CPU is interrupted when the TX FIFO has been drained to a low water mark set in **tx_lwm**. The constraint on this register is (1<= **tx_lwm** <= 16). An interrupt is issued if the number of available spaces in the FIFO has fallen below the water mark. In addition, the Tx is stopped from sending a new byte when the input signal nUARTCTS is asserted low.

  The hardware flow control signals can be overridden with software-controlled registers **hw_flow_ovrd**. Furthermore, for debugging purposes, the user can:

  - find the number of free spaces in the RX FIFO and TX FIFO in **rx_fifo_spaces** and **tx_fifo_spaces** respectively.
  - see the status of the hardware flow control signals in bit 5 of **state** register for nUARTCTS and bit 4 for nUARTRTS.

- **PDM Digital microphone**

  Pulse Density Modulation (PDM) provides microphone support using two pins and supports three clock speeds

CONFIDENTIAL

(500 kHz, 1 MHz, and 2 MHz). The PDM-to-PCM conversion block takes as input a 1-bit PDM output from a digital microphone and outputs a 16-bit PCM signal at 16 kHz sample rate. The clock frequency to the PDM microphone can be selected to be 500 kHz, 1 MHz or 2 MHz. The 1-bit PDM signal is filtered through a 3rd order sinc decimation filter to filter out ADC quantization noise. This is followed up by a 1st order IIR filter (that can be bypassed) which attenuates any residual DC content. The corner frequency of the IIR filter is programmable. The gain control block has a programmable gain from -30 dB to 30 dB in steps of 0.5 dB. The output bit rate is 16 bit x 16 kHz = 256 kbps.

The core at_apb_pdm is an APB slave with two separate sets of FIFO arrays one for APB, one for AHB (faster) access:

1. APB 16-entry FIFO which overruns quickly if not drained faster
2. AHB FIFO implemented using ping-pong buffer (32x32). 32 entries are split into 2 arrays - bufferA, bufferB. PDM output (16 bits) are written in bufferA, then written to bufferB. AHB access is faster and data can be DMA'd into SRAM or FLASH-based on application.

- **PWM**

  The Pulse Width Modulation (PWM) has eight independent output channels. Each channel has frequency and duty cycle options. Additionally, all channels can be started at exactly the same time if synchronization is required. The frequency ranges from 250 kHz to 8 MHz and duty cycle is tunable in 1/64th steps.The core is an APB slave and it provides pulse width modulation functionality with up to eight output channels and configuration of these channels is done through register programming over APB. Each of the eight channels are identical but have their own configuration register.

  For each enabled channel, the FSM runs through a (*tot_dur*+1) period sequence. For the first portion, the PWM output signal will be high. For the last *lo_dur* periods, the PWM output signal will be low. A channel is enabled by setting the *ok_to_run* field in the corresponding control register. If channels need to be started simultaneously, the field *alt_ok_to_run* in register **pwm0_ctrl** should instead be used. Each channel can optionally be inverted. The field *tot_dur* is 6 bits so the maximum period is 64 clocks cycles which at a base clock of 16 MHz is 250 kHz. The field *lo_dir* is also 6 bits so the duty cycle is tunable to a 1.6% (1/64) step size.

- **Quadrature decoder for mouse inputs (QDEC)**

  The QDEC is a 3-axis (x,y,z) quadrature decoder. Each decoder is independent and requires 2 input pins. QDEC continuously updates the internal integrators and their values can be read by software.

  The core is an APB slave and supports QDEC with 3-axis (x,y,z) in multiple configurations. Each axis requires 2-input pins which can be setup using **pin_selection_**x registers in WRPR.

  QDEC provides the following registers per axis:

  - Setup register (polarity, purge values)
  - QDEC core accumulates the individual axis movements and writes into accumulator registers.
  - Error values are logged in error count registers
  - Accumulator, error purge to clear out accumulator, error values respectively.

  To setup QDEC use following steps:

  - Enable clock, toggle reset to QDEC core by writing to **apb12_ctrl** register in WRPR.
  - Read qdec *core_id* and it should match 32'h51444543 (defined in register spec).
  - Setup qdec pins by writing to WRPR **pin_selection_**x registers. You need to setup all 3-axis.
  - Set polarity, purge bits in **ctrl0** register.
  - Read accumulator, error register per axis to compute the direction, spin.
  - Clear registers if required.

- **Keyboard matrix controller (KSM)**

  The KSM is a keyboard scanner which supports multiple keyboard sizes up to 8 rows by 20 columns or 20 rows by

CONFIDENTIAL

8 columns for the ATM2201/ATM2221/ATM3201/ATM3221/ATM3231, and up to 3 rows by 7 columns or 7 rows by 3 columns for the ATM3202/ATM2202/ATM2231, which can be setup using **pin_selection_**x registers in WRPR. Key pressed and key released events can optionally require that the key be pressed or be released across multiple consecutive scans. Up to 12 simultaneous key events can be tracked.
A portion of the KSM can continue to scan during low power states. The key pressed or key released event can also optionally wake the ATM2/ATM3 from a low power state. Key events are not lost when the ATM2/ATM3 is in a low power state. Key events are packetized and written into a FIFO via the hardware for later reading by the software.
To setup KSM use following steps:

- Enable clock, toggle reset to KSM core(s) by writing to **apb10_ctrl** register in WRPR.
- Read KSM **core_id** and it should match 32'h57525052 (defined in register specification).
- Setup KSM pins by writing to **pin_selection** registers in at_apb_wrpr.
- Setup **matrix_size**, **time_param0**, and **time_param1** registers.
- Set *go*, *consecscan* bits in **ctrl0** register, this initiates KSM scan and starts key capture.
- There are 2 ways to signal transaction end
    - Polling - Read status using **keyboard_packet** register *empty* bit.
    - Interrupt - KSM core generates interrupt which can be used by M0. Interrupt can be masked, set, cleared with appropriate registers
- FIFO supports 2 interrupts (data, full) and they can be set up individually.
- Read **keyboard packet** register which contains pressed/released bit, *row*/*col* entry.

- **Analog Comparator**
  A 16-level analog comparator provides an ultra-low power approach to sense an analog input signal from sensors.

CONFIDENTIAL

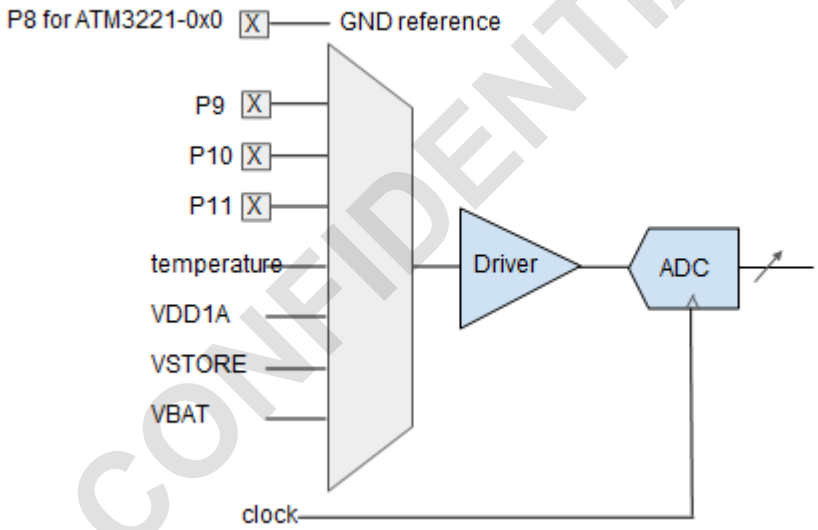
2.10 Application ADC

The Application ADC, also known as the general-purpose ADC (gADC), enables applications to digitize an external analog signal or certain internal signals (temperature, VDD1A, VSTORE, VBAT). P9, P10, or P11 can be configured to be single-ended inputs while P10 and P11 can be configured to be a differential input.

When digitizing a single-ended input (including VDD1A, VSTORE, and VBAT), a ground reference is needed. P8 is used as an external ground reference for ATM3221-x0x. As a result, P8 must be left unconnected for ATM3221-x0x applications that need it as a ground reference. In the current SDK, the ATM3221-x0x keyboard example includes P8 in the keyboard matrix, so its VBAT readings are invalid. An internal ground reference is used for ATM3221-x1x, so ATM3221-x1x applications have no limitations on using P8. Please refer to Atmosic Chip Revisions Application Note for information regarding chip revisions.

The temperature sensor is sensitive to VDDIO/VDDIOA, so temperature measurement is only supported for 1.8 V IO supply.

Figure 2.10-1 Application ADC block diagram



CONFIDENTIAL

Table 2.10-1 Application ADC Electrical Parameters

Parameter	Value	Comments
Resolution	10 bits	
Current consumption	VBAT at 3V: 15 µA VDDIO/VDDIOA at 1.8 V: 20 µA	Typical values at room temperature
Sampling rate	2 MHz	Does not include hardware or software averaging
Temperature measurement accuracy	+/-5 °C	Typical value at room temperature
Voltage measurement gain error	+/-1 %	Typical value at room temperature[4]
Voltage measurement offset error	+/-2 LSB	Typical value at room temperature[4]

[4] Common mode voltage set to 0.5 * VDDIO/VDDIOA for differential input.

Each channel is one of the possible combinations of actual input mux selection. Gain in the signal path is different for each channel, and can be controlled by one bit control *gext* in the gADC **final_inversion** register. The gain settings and corresponding input ranges for each channel are summarized in Table 2.10-2.

Table 2.10-2 Summary of ADC Channels

watch_channels	gext	Description	Range (min, max)	Resolution (mV/LSB)
0	0	VBAT 3.3 V	1.0, 3.3	4.7
	1	VBAT 1.8 V	1.0, 1.8	2.3
1	0	VSTORE 3.3 V	0, 3.3	6.2
	1	VSTORE 1.8 V	0, 1.8	3.1
2	0	VDD1A	1.000, 1.024	1.6
	1	N/A	N/A	N/A
3	0	Temperature	-40, 85	N/A
	1	N/A	N/A	N/A
4	0	P10/P11 in differential mode 3.3 V	-3.3, 3.3[5]	7.8
	1	P10/P11 in differential mode 1.8 V	-1.8, 1.8[5]	3.9
6	0	P10 in single-ended mode 3.3 V	0, 3.3[5]	7.8
	1	P10 in single-ended mode 1.8 V	0, 1.8[5]	3.9
7	0	P11 in single-ended mode 3.3 V	0, 3.3[5]	7.8
	1	P11 in single-ended mode 1.8 V	0, 1.8[5]	3.9
9	0	P9 in single-ended mode 3.3 V	0, 3.3[5]	7.8
	1	P9 in single-ended mode 1.8 V	0, 1.8[5]	3.9

CONFIDENTIAL

From the SoC point of view, the Application ADC (gADC) core is an APB slave. This core generates the gADC control signals; applies offset, gain, and averaging functions; and contains a FIFO for the CPU to get access to the post processed data.

For the digitizing flow to work, the gADC needs to first be powered up. This is done via the PSEQ **config** register. The steps are:

1. Short the supply head switch via **gadc_config**.*gadc_cutvdd_b*
2. Then release reset and set the enable via **gadc_config**.*gadc_rstb* and **gadc_config**.*gadc_en*.

The GADC channels to be monitored are selected via GADC **ctrl**.*watch_channels* with programmable gain setting of **final_inversion**.gext. The description of each (**ctrl**.watch_channels, **final_inversion**.gext) are in each row of watch_channels and gext in the table.

The core contains a single output interrupt with different sources. Each source is maskable, clearable, and settable independently. The interrupt sources are:

- Hardware written to FIFO, may be of use in cases where **ctrl**.wait_amount is long enough for the CPU to pull from the FIFO before gADC writes to FIFO.
- FIFO overrun.

5 Input pin voltage should not exceed VDDIO/VDDIOA.

The samples in the FIFO contain both the channel ID and the ADC sample. The FIFO sample is 16-bit wide with the upper nibble being the channel ID and the lower 12 bits being the averaged ADC sample. The ADC sample in FIFO is signed.

To convert raw ADC samples to real values (voltage or temperature), the gADC software driver in the SDK measures the ground reference as needed, averages a fixed number of samples, and applies chip-specific calibration data that is stored in OTP. The complete procedure of making a measurement in software can take up to approximately 300 µs.

2.11 Register Specification Document

The various registers of ATM2/ATM3 are documented in the Register Specification Document.

- TIMER0 - Timer 0
- TIMER1 - Timer 1
- DUALTIMER2
- WRPR - Write Privileged Register
- UART0
- UART1
- PWM
- SPI0
- WATCHDOG
- KSM
- SPI2
- QDEC
- SPI1
- SLWTIMER - Slow Timer
- QSPI - Quad SPI
- GPIO0
- GPIO1
- I2C0
- I2C1
- NVM - OTP
- WRPR1
- RADIO
- PMU
- PSEQ - Power Sequencer
- GADC - General Purpose or Application ADC
- TRNG - True Random Number Generator
- RCOS_CAL - RC Oscillator Calibration
- SHUB - Sensor Hub
- PDM
- AT_DMA
- PMU_TOP
- PMU_WURX

CONFIDENTIAL



CONFIDENTIAL

ATMOSIC TECHNOLOGIES - DISCLAIMER

This product document is intended to be a general informational aid and not a substitute for any literature or labeling accompanying your purchase of the Atmosic product. Atmosic reserves the right to amend its product literature at any time without notice and for any reason, including to improve product design or function. While Atmosic strives to make its documents accurate and current, Atmosic makes no warranty or representation that the information contained in this document is completely accurate, and Atmosic hereby disclaims (i) any and all liability for any errors or inaccuracies contained in any document or in any other product literature and any damages or lost profits resulting therefrom; (ii) any and all liability and responsibility for any action you take or fail to take based on the information contained in this document; and (iii) any and all implied warranties which may attach to this document, including warranties of fitness for particular purpose, non-infringement and merchantability. Consequently, you assume all risk in your use of this document, the Atmosic product, and in any action you take or fail to take based upon the information in this document. Any statements in this document in regard to the suitability of an Atmosic product for certain types of applications are based on Atmosic's general knowledge of typical requirements in generic applications and are not binding statements about the suitability of Atmosic products for any particular application. It is your responsibility as the customer to validate that a particular Atmosic product is suitable for use in a particular application. All content in this document is proprietary, copyrighted, and owned or licensed by Atmosic, and any unauthorized use of content or trademarks contained herein is strictly prohibited.

Copyright ©2019-2022 by Atmosic Technologies. All rights reserved.

www.atmosic.com